

National University of Ho Chi Minh City
HCM University of Technology
Computer Science and Engineering Department



Report: Specialized Project (CO4029)

SmartEdu

**A Graph-based Intelligent Teaching Assistant
for Mastery-oriented Learning Roadmap
Guidance and Supervision**

Major: Computer Science

Council:

Instructor: Assoc. Prof. Thoại Nam

Secretary:

—o0o—

Student: Vũ Hoàng Tùng (2252886)

HỒ CHÍ MINH City, June 2026

Acknowledgement

This section is dedicated to express our sincere gratitude towards all those who have played a crucial role in the completion of the Specialized Project, also the first phase of the thesis. Your supports have been a great help for me to perform various steps from the fundamental researches and requirement specification to system design and implementation development, laying foundations for the future Capstone Project

I would like to show my foremost appreciation to my supervisor, Mr.Thoai Nam for his unwavering support, guidance and constructive feedback. His invaluable expertise and advice has really strengthened my vision and understanding, thus been instrumental in shaping the direction of my work.

I am also greatly thankful to Computer Science Faculty of Department for the learning environment, the facilities, the knowledge and the opportunity to orchestrate this work that I'm proud of. The contributions of my peers and colleagues in engaging discussions and shared insights have been enriching.

To Bảo, a colleague and a friend of mine, whose knowledge and engaging discussion on system design has been a great help, I show my appreciation for his passionate and knowledgeable share and consistent heartwarming support.

Finally, to all my family and friends who offered encouragement, shared their knowledge and gave inspiration, I feel blessed that this Project and me have been supported by your love and care.

Last but not least, I acknowledge that this my work would not achieve the goal without the foundation laid by pioneering researchers and engineers, both in the field of artificial intelligence and education. The theories, methodologies, and technologies they have developed have been the bedrock upon which this project is built. The knowledge they have contributed is invaluable, and I am more than honored to stand on the shoulders of giants.

This work stands firm thanks to the contribution of all those mentioned above. Your accumulated impact has been indispensable, and I cherish the roles each played in this academic endeavor.

DECLARATION OF AUTHENTICITY

I guarantee that this specialized project entitled "*SmartEdu: A Graph-based Intelligent Teaching Assistant for Mastery-oriented Learning Roadmap Guidance and Supervision*" was entirely composed and implemented by myself under the guidance and supervision of Assoc. Prof. Thoại Nam at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

Throughout the project, I ensure that all references and external sources utilized are appropriately cited and thoroughly documented. Although full code won't be included in report, important parts of the code will be summarized in pseudo form and referenced to source of inspiration (if yes).

Ho Chi Minh City, May 2026
The author,

Vũ Hoàng Tùng

Abstract

I like to acknowledge ...

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	2
1.3	Scope	2
2	Fundamental Knowledge & Related Work	4
2.1	Fundamental Knowledge	4
2.1.1	Natural Language Processing	4
2.1.1.1	Definition	4
2.1.1.2	Information Extraction (IE)	4
2.1.1.3	Knowledge Modeling	5
2.1.1.4	Information Retrieval (IR)	5
2.1.2	Large Language Models (LLMs)	6
2.1.2.1	Definition and Core Architecture	6
2.1.2.2	LLM Ecosystem: Proprietary vs Open-Weights	6
2.1.3	Agentic Systems	7
2.1.3.1	AI Agents: Definition and Capabilities	7
2.1.3.2	Agent Frameworks	8
2.1.3.3	Langchain and LangGraph	9
2.1.4	System Design	10
2.1.4.1	Software Paradigm: Object-Oriented Programming	10
2.1.4.2	System Architecture: Microservices	11
2.1.5	Technology Stack and Frameworks	12
2.1.5.1	Backend Development	12
2.1.5.2	Frontend Development	12
2.1.5.3	Ollama: Local Model Inference and Optimization Infrastructure	13
2.1.5.4	The Langchain Ecosystem	14
2.1.6	Database System Design & Management	14
2.1.6.1	Relational Database (RDBMS)	14
2.1.6.2	Non-Relational Database (NoSQL)	15
2.1.6.3	High-Performance Object Storage (MinIO)	16
2.1.6.4	Vector Database	16
2.1.6.5	Graph Database	17
2.1.6.6	Database Management and Observation	17
2.1.7	Deployment and Containerization	17
2.1.7.1	Docker	17
2.1.7.2	AWS	18
2.1.8	Security: OAuth 2.0 Authorization	19
2.2	Related Work	19
2.2.1	Document-Knowledge Modeling	19
2.2.2	Generative AI and Agentic Systems in Education	20
2.2.2.1	Advances in Educational Agents	20
2.2.2.2	Challenges and Deployment Bottlenecks	20
2.2.2.3	Verifiable Evaluation of Agentic Workflows	21
2.2.3	Educational Intelligent System	21
2.2.4	Conclusion and Research Gaps	22

3	Requirements Specifications	23
3.1	Functional Requirements	23
3.1.1	Knowledge Modelling and Management	23
3.1.2	Intelligent Teaching Assistance	24
3.1.3	Student Interaction and System Navigation	24
3.2	Non-Functional Requirements	25
4	Proposed Methodology and Paradigm	26
4.1	Overall Pipeline	26
4.2	Knowledge Modelling: Educational Graph	26
4.2.1	Graph Definition	26
4.2.1.1	Formal Property and Topological Definition	27
4.2.1.2	Ontological Hierarchy formulation	27
4.2.1.3	Edge Semantics and DAG Constraints	27
4.2.2	Graph Construction	28
4.2.2.1	Dual Phase LLM Extraction	28
4.2.2.2	NLP-Based Concept Filtering and Verification	29
4.3	Teaching Assistance	30
4.3.1	Agent Harnessing	30
4.3.1.1	Tool Abstraction Design	32
4.3.1.2	State—based Harnessing	32
4.3.1.3	LangChain Middleware Architecture	32
4.3.1.4	Agentic Cooperation into Workflow	33
4.3.2	Agentic Workflow	34
4.3.2.1	Workflow: Retrieve	35
4.3.2.2	Workflow: Roadmap	36
4.3.2.3	Workflow: Teach	38
5	System design and implementation	42
5.1	System Architecture	42
5.1.1	Overall Architecture and Module Responsibilities	42
5.1.2	Singleton via Lifespan Management	43
5.1.3	Dependency Injection via <code>core/dependencies.py</code>	44
5.1.4	MVC-Aligned Module Structure and Scalability to Microservices	44
5.2	Core Module	44
5.2.1	LLM Engine: Multiton and Profile-Based Configuration	44
5.2.2	Configuration Management: Dataclass-Based Centralization	45
5.2.3	Shared Schema: The System Lingua Franca	45
5.2.4	Repository Clients: Abstracted Database Access	45
5.3	Student Module	45
5.3.1	Authentication: OAuth2 + JWT	45
5.3.2	Session Lifecycle and the Identity Firewall	46
5.3.3	Student_Tracker: Multi-Session Isolation and Shared State	46
5.4	Knowledge Module	46
5.4.1	Ingestion Pipeline	46
5.4.2	Graph Schema	46
5.5	Teaching Assistant (TA) Module	47
5.5.1	Module-Level Design: Stateless Singleton with External Context	47
5.5.2	Agent Injection: Factory + Strategy Patterns	47
5.5.3	SmartEdu: State Machine (LangGraph)	47
5.5.4	Teaching Sub-Graph: Determinism and RAG Fallback	47
5.5.5	Prompt Management: Module-Level <code>__getattr__</code>	48
5.6	Tracing and Observability Module	48
5.7	Database Design and Management	48

List of Figures

2.1	Naive Bayes. Calculate probability of next token given n previous tokens using chain rule	5
2.2	Comparison of LLM Deployment Strategies: Traditional approaches (a) Direct cloud processing, (b) Local-only processing, and (c) Local-Cloud framework balance performance security without leaking IP (proposed by [17])	6
2.3	Basic Single Agent diagram	7
2.4	Comparison of Microservice and Monolithic architecture	11
2.5	ReAct Virtual DOM; Source: GeeksforGeeks	13
2.6	Architecture of the EDC framework for extracting and canonicalizing knowledge triples.	19
2.7	The end-to-end EduKG Pipeline framework for educational knowledge graph construction.	20
2.8	Dynamic Knowledge Graph (DKG) approach tracking knowledge evolution over time.	20
4.1	Macro workflow of the SmartEdu system, illustrating the Knowledge Modelling, Middleware, and Agentic Workflow orchestration.	26
4.2	Dual Phase Extraction	28
4.3	Agent Harness architecture demonstrating the Adaptive Harnessing based on Middleware.	31
4.4	Langchain Middleware. Referenced from Langchain Documents	33
4.5	How agents are cooperate into each node of the workflow	34
4.6	Top-level SmartEdu state machine detailing the hierarchical intent routing and background persistence synchronization.	34
4.7	Retrieval sub-graph. The flow routes dynamically to deep verification of prerequisite concepts if potential knowledge gaps are identified.	35
4.8	Roadmap sub-graph. The proposed path is queried from the EKG, critiqued against the student's mastery profile, and generated with bespoke pedagogical advice.	37
4.9	Teaching sub-graph flow detailing deterministic content lookup, RAG fallback pathways, active instruction delivery, and adaptive topic progression.	39
5.1	Modular Architecture Overview of SmartEdu	43
5.2	Ownership graph in lifespan. Shared resources are injected into domain modules	43
5.3	Top-level SmartEdu state machine. Each WF_* node is itself a compiled sub-graph.	47
5.4	Teaching sub-graph. The conditional edge from Teach_Lookup is resolved by a Python function, not an LLM call.	48

List of Tables

2.1	Comparison of LLM Orchestration Frameworks	10
2.2	Polyglot Persistence Architecture: Strategic allocation of storage engines.	14
2.3	Comparative analysis of representative educational intelligent systems based on key AI and knowl- edge features.	22
5.1	System Module Summary	42
5.2	Shared Schema Classes in <code>core/schema/wf_state.py</code>	45
5.3	Polyglot Persistence Strategy	49

Chapter 1

Introduction

1.1 Motivation

The rapid evolution of Artificial Intelligence (AI) has fundamentally change the global socio-economic landscape. To adapt to the new era, selfstudying and learning method are becoming more and more important. Traditional education philosophy, such as "practice makes perfect," has no longer been the absolute true; when AI can automate execution, the value of learning moves from procedural fluency and labour expertise to conceptual mastery and strategic analysis. However, current education system fail to keep pace with this evolution:

- **Stagnancy of Content:** Legacy Learning Management Systems (LMS) treat knowledge as static artifacts (textbooks, slides). These materials fail to adapt to the varying cognitive baselines of individual students.
- **The Guidance Deficit:** Without a dynamic roadmap, students navigate a "sea of information" where external resources are either too granular (leading to information overload) or too condensed (creating knowledge gaps), with little profound ground truth.
- **The Lack of Interaction:** Current systems lack a feedback loop. Textbooks exceed the average human attention span, while slides provide too condensed context insufficient for novices, leaving no room for real-time, human-like inquiry or mentorship.
- **Outdated Knowledge:** Educational materials and methods cannot catch up with the change of technology. Internet are an invaluable resource, however information is chaotic, unstructured and content are confusing to many new learners.

While Large Language Models (LLMs) offer a glimpse into automated resource synthesis and knowledge base construction, significant technical bottleneck prevent their reliable application in complex domains:

- **Semantic Fragmentation in RAG:** Traditional Retrieval-Augmented Generation (RAG) frameworks rely on discrete data chunking. This fragmentation disrupts the global structural integrity of a subject, leading to "contextual myopia" where the model captures isolated facts but fails to comprehend the overarching hierarchy of the domain.
- **Contextual Saturation vs. Precision:** Expanding the context window of LLMs often introduces diminishing returns. The injection of unrefined data frequently leads to "contextual overhead" and "noise," where the model struggles to distinguish fundamental principles from peripheral details.
- **Relational Limitations of Vector Space:** Relying exclusively on vector embeddings (typically ≈ 1024 dimensions) is insufficient for mapping the non-linear, multi-faceted dependencies inherent in academic taxonomies. Similarity-based retrieval cannot substitute for the deterministic logic required to understand prerequisite relationships.
- **The Reasoning Gap in Pedagogy:** Instruction is a sophisticated, non-linear reasoning task. Current AI systems are largely reactive—proficient in text generation but deficient in long-term goal tracking and proactive guidance. This results in a susceptibility to hallucinations and an inability to execute the "evaluate-and-guide" workflow essential for effective pedagogy.

1.2 Goals

To address the current limitations, this project aims to design an Intelligent System far beyond serving contents or a simple QA chatbot: a proactive, mastery-oriented learning platform tracking and adapting to student progress by developing **SmartEdu**—an intelligent, graph-grounded educational ecosystem.. This thesis determined for such a system which capabilities include:

- **Adaptive Mentorship:** Shifting the educational paradigm from static, linear content delivery systems to an interactive, human-like mentorship that dynamically reform itself based on student performance.
- **Data Sovereignty and Localized Execution:** Establishing a sustainable deployment framework that leverages open-weights Small Language Models (SLMs) and local inference engines, ensuring complete data privacy for student learning behaviors while eliminating recurring API token costs.
- **Consistent Educational Contents:** Intelligent contents adaptation and generation suffers from hallucinations and key info lost. Mitigating the hallucination risks inherent in generative AI by anchoring the multi-agent reasoning framework within a structured knowledge network to enforce academic truthfulness.

Targeted at University students, this educational platform facilitates student self learning, the most important skills in University. To demonstrate practical utility and fulfill the discussed goals of an actionable software solution, the application identifies the following key features:

- **Automated Educational Knowledge Graph (EKG) Engine:** The system automatically ingests unstructured academic materials (raw PDFs, textbooks, and lecture videos) and constructs a structured knowledge graph. It automatically extracts conceptual hierarchies, maps prerequisite dependencies, and group related topics into organized clusters.
- **Dynamic Student Mastery Mapping:** The application implements a continuous storing-and-tracking engine that tracks the student's real-time comprehension levels, storing historical interaction and cognitive progress securely.
- **Automated Gap Detection and Pathfinding:** Leveraging graph-traversal and heuristic algorithms, the system choose a good studying path based on the student's current mastery and state against the EKG backbone. When a target topic is selected, the application automatically identifies missing prerequisite knowledge and dynamically generates a personalized, non-linear learning path.
- **Socratic Dialogue and Document Verification:** The user interface provides an interactive teaching assistant that engages students using powerfull reasoning workflow.

1.3 Scope

To fulfill these goals, the research focuses on the following researching scope:

- **Multimodal Information Extraction:** Knowledge of courses from multiple sources (pdf, videos, textbooks) must be well extracted to consolidate them into a centralized data repository.
- **Domain Specificity:** The knowledge modeling and extraction processes are limited to well-defined academic disciplines with clear hierarchical structures. Unstructured or subjective domains that lack clear prerequisite dependencies are outside the scope of this research.
- **Pedagogical Boundaries:** The conversational assistant is strictly restricted to goal-oriented educational support, specifically for assessing student understanding and explaining course content. Open-domain dialogue, creative writing, and casual chitchat are intentionally disabled.
- **Infrastructure:** The system optimizes execution using open-weights models to maintain strict data privacy.
- **Target User Environment:** The system is modeled and evaluated for a single-institution environment, focusing on the interactions between individual learners and a localized knowledge base, rather than a multi-tenant, global-scale educational network.

There are 7 chapters in this project's report:

- **Chapter 1** introduces the problems, proposes the system's goals, scopes and thesis structure
- **Chapter 2** provides fundamental knowledge for project understand and deep literature review of relevant prior researches covering topic, as well as preview of open-source application.

- **Chapter 3** Requirement Specifications
- **Chapter 4** propose methodologies: we discuss what inferred from previous work and what ways to solve the specific usecase of ours
- **Chapter 5** details the system including: System architecture and its components, settings in each components, and finally, their implementation explanation that fully illustrate how do the system put proposed methodology to practice
- **Chapter 6** analyzes the system outcomes from components to final output as a whole
- **Chapter 7** is the conclusion to the result: what the system achieved, its limitation and future improvements for the final Capstone Project

Chapter 2

Fundamental Knowledge & Related Work

2.1 Fundamental Knowledge

This section discusses the fundamental techniques and applications foundational to the construction of the proposed system, categorized into 3 pillars:

- **Natural Language Processing (NLP):** The foundational process of transforming vast volumes of raw, unstructured textual input into logical, structured data formats that machine can understand. This is fundamental for document processing, reshaping all subsequent cognitive actions.
- **Large Language Models (LLMs):** The core reasoning and action components of the system, responsible for interpreting user queries, orchestrating tools, and generating responses. This pillar represents the "intelligence" of the system, enabling it to perform complex tasks and adapt to user needs.
- **Agentic Orchestration:** The management layer responsible for coordinating specialized AI agents to execute complex, non-linear workflows.
- **System Design:** The architectural blueprint that ensures scalability and robustness, utilizing clean design patterns tailored for high-stakes educational environments.

2.1.1 Natural Language Processing

2.1.1.1 Definition

Natural Language Processing (NLP) is a crucial field of study aiming to process unstructured data from documents to a structured, semantically rich representation. This process is designed to harness the full potential of complex entity-relationship paradigms, moving from raw text to machine-inferable logic. As outlined in the core objectives of language understanding, the goal is to put information into a semantically precise form that allows for further inferences.

The standard pipeline for Natural Language Processing follows the sequence:

Information Extraction (IE) → Knowledge Modeling → Information Retrieval

2.1.1.2 Information Extraction (IE)

Information Extraction is the task of identifying and understanding limited, relevant portions of a text to produce a structured representation [6]. Its goal is to filter out linguistic noise and isolate the atomic units of knowledge. Traditional algorithms aim to extract keywords, following the 2 methods:

- **Keyword and Feature Extraction:** Utilizing dependency parsing and statistical methods (e.g., TF-IDF) to identify high-signal terms based on syntactic importance.
- **Named Entity Recognition (NER):** A critical sub-task used to find and classify concepts or entities (e.g., specific academic theories, formulas, or historical figures) within a text database [6]. This identifies the "nodes" of our future backbone.

To capture complex entities and relationships that rigid rule-based systems miss, the process relies on probabilistic modeling. The foundation of this approach is Tokenization, which breaks down raw text into atomic units (tokens). Statistical algorithms, such as Conditional Random Fields (CRF) or Hidden Markov Models (HMM), are then

applied to these tokens to calculate the probability distribution of various interpretations. This allows the system to resolve linguistic ambiguities and predict the most likely semantic structures or labels for sequences of text.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 2.1: Naive Bayes. Calculate probability of next token given n previous tokens using chain rule

While traditional probabilistic models are effective, they often struggle with sparse data and lack deep semantic understanding. Deep Learning represents a significant advancement by shifting the target towards generating Embeddings. Instead of merely calculating discrete probabilities, Deep Learning architectures map tokens into a continuous, high-dimensional vector space. In this embedding space, geometrically close vectors represent semantically related concepts, allowing the system to grasp the underlying meaning, context, and nuance of the information rather than relying solely on surface-level statistics. The first major breakthrough in applying Deep Learning to text came with Sequential Neural Networks (SNNs), specifically Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. These models process tokens sequentially—from left to right—maintaining a hidden state that acts as a memory of previous tokens. This sequential processing is crucial for understanding the context of a word based on what preceded it. However, SNNs face limitations in capturing long-term dependencies due to vanishing gradients and are computationally inefficient because they cannot process tokens in parallel.

The Transformer architecture was introduced, overcoming the drawbacks of SNN and being SOTAs as well as standard method nowadays. It fundamentally change the landscape of NLP and enable modern Large Language Models (LLMs). The core innovation of the Transformer is the Attention Mechanism (specifically, Self-Attention). Instead of processing tokens sequentially, Transformers process the entire sequence simultaneously. The Attention Mechanism calculates a weighted importance score for every token relative to every other token in the sequence, regardless of their distance. This allows the model to instantly capture global context and long-range dependencies, making it the most powerful method for extracting deep, complex information from educational texts.

2.1.1.3 Knowledge Modeling

Often overlooked in standard RAG systems, Knowledge Modeling is the construction stage where extracted entities are organized into a coherent structure that shapes how the system perceives and reasons.

- **Semantic Parsing and Logical Form:** Mapping extracted relations into a semantically precise form, such as First-Order Predicate Calculus (FOPC) or Knowledge Graphs (KG). This creates a deterministic "Backbone Graph" of the curriculum.
- **Embedding Space Topology:** Modeling knowledge within a continuous vector space where semantic similarity is represented by geometric proximity. This allows the system to handle concepts that are conceptually related but lexically distinct.

2.1.1.4 Information Retrieval (IR)

Information Retrieval is the active execution stage based on the result of construction phases IE and Knowledge Modeling, where the goal is to identify documents or data points relevant to a specific information need from a large document set [6].

- **Lexical and Keyword Matching:** Corresponding to traditional IE, this method uses exact term overlaps to find specific references or symbolic links within the knowledge base.
- **Vector Retrieval:** the most common modern approach, utilizing the embedding space to retrieve context based on semantic similarity, typically calculated via Cosine Similarity:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} \quad (2.1)$$

- **The GraphRAG Paradigm:** A new powerful emerging method, where retrieval navigates the entity-relationship graph established during the modeling phase. Unlike discrete chunk retrieval, GraphRAG performs "multi-hop" reasoning to connect disparate concepts, enabling the system to resolve complex queries that require a global understanding of the educational roadmap.

2.1.2 Large Language Models (LLMs)

2.1.2.1 Definition and Core Architecture

Large Language Models (LLMs) are deep learning-based probabilistic models trained on massive datasets to understand and generate human-like text. Mathematically, an LLM performs the task of next-token prediction by modeling the conditional probability distribution of a token w_t given a sequence of preceding tokens $(w_1, w_2, \dots, w_{t-1})$:

$$P(w_t | w_1, w_2, \dots, w_{t-1}) \quad (2.2)$$

The structural foundation of modern LLMs is the **Transformer** architecture, which relies on the **Self-Attention** mechanism. This mechanism allows the model to compute the relative importance of different tokens in a sequence regardless of their distance. The attention score is calculated using Query (Q), Key (K), and Value (V) matrices derived from input embeddings:

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.3)$$

By utilizing multi-head attention and positional encoding, Transformers can capture complex linguistic patterns and long-range dependencies more effectively than previous sequential architectures like Recurrent Neural Networks (RNNs).

[Image of Transformer architecture diagram]

2.1.2.2 LLM Ecosystem: Proprietary vs Open-Weights

The landscape of Large Language Models in the industrial layer (not research layer) is categorized by deployment strategies and the degree of architectural transparency. Closed-source and cloud-based models, such as GPT-4 and Claude, represent the proprietary tier of this ecosystem. These models are typically provided as a Service (SaaS), where interactions occur via Application Programming Interfaces (APIs) hosted on centralized, high-performance infrastructures. While these proprietary solutions consistently deliver state-of-the-art performance across a wide range of benchmarks, their internal mechanisms—including model weights, training datasets, and specific configurations—remain opaque. Furthermore, their operational use often involves per-token costs, which can become a significant factor in large-scale or iterative research applications.

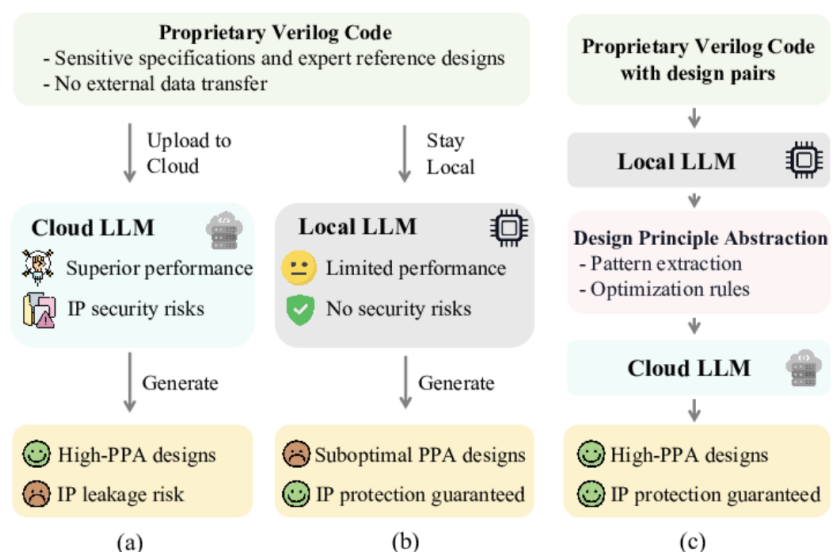


Figure 2.2: Comparison of LLM Deployment Strategies: Traditional approaches (a) Direct cloud processing, (b) Local-only processing, and (c) Local-Cloud framework balance performance security without leaking IP (proposed by [17])

In contrast to the proprietary paradigm, the emergence of open-weights and local models has introduced a significant shift toward decentralized execution and architectural transparency. Initiatives such as Meta's Llama series and Mistral release their pre-trained model parameters to the public, allowing for deployment on private, local infrastructures. This transparency facilitates full control over data sovereignty and privacy, as information does not need to leave the local environment to be processed. Consequently, these models serve as foundational baselines, providing a stable platform for both academic research and industrial innovation without the necessity of external API dependencies or recurring subscription fees.

Building upon these general-purpose baselines, recent advancements in the field have catalyzed a trend toward specialization, where models are fine-tuned or structurally modified for high-performance niche tasks. One prominent example is the development of coding specialists, such as Qwen-coder or CodeLlama, which are optimized specifically for source code synthesis and complex logical reasoning. These specialized variants often rival or even outperform significantly larger general-purpose models in programming-centric contexts. Beyond text processing, research has also extended these baseline architectures into the multimodal and generative domains. By integrating vision encoders for visual comprehension or combining LLM backbones with diffusion-based architectures, researchers have developed sophisticated models capable of high-fidelity image generation and cross-modal reasoning.

2.1.3 Agentic Systems

2.1.3.1 AI Agents: Definition and Capabilities

An Artificial Intelligence (AI) Agent is a computational system capable of autonomous task execution through the dynamic design of workflows using available tools. AI agents can encompass a wide range of functions beyond natural language processing including decision-making, problem-solving, interacting with external environments, and performing actions.

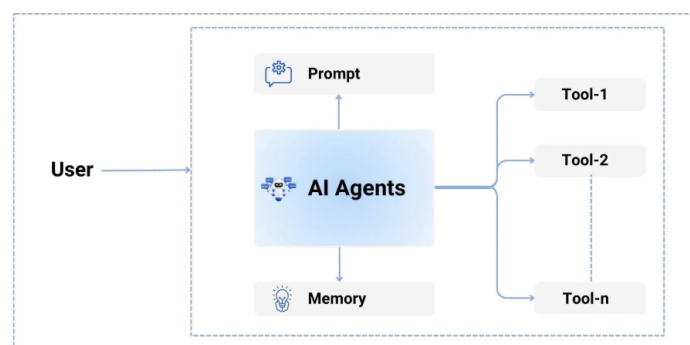


Figure 2.3: Basic Single Agent diagram

Unlike Large Language Models (LLMs), which produce responses solely based on the static data used to train them and are bounded by fixed knowledge limits, agentic technology uses tool calling on the backend to obtain up-to-date information. They optimize workflows and create subtasks autonomously to achieve complex goals. It is responsible for translating high-level natural language queries into executable workflows. Instead of relying on rigid, pre-programmed paths, the agent dynamically analyzes the request, identifies the necessary tools, and orchestrates them in a sequence that best addresses the user's needs. This allows for a more flexible and adaptive system that can handle a wide variety of queries and tasks without requiring explicit programming for each scenario.

What truly pushing agentic systems to achieve what LLM can't lies in the harnessing of agentic technology. Harness is the process of constructing and integrating systems or infrastructure that boosts capabilities into the system. The harnessing of agentic system is critical for the agentic performance, as it enables the system to navigate the complex, multi-step processes required to analyze and retrieve relevant educational content based on user queries.

LLM: The Logical Core Large Language Models (LLMs) serve as the brain of the agent, represent the thinking capability. They provide the reasoning capabilities necessary to interpret vague user queries and orchestrate the appropriate tools to find answers. The LLM is responsible for parsing the user's intent, identifying missing information, and deciding on the next actions. The reasoning and decision-making depend heavily on the type of LLM used. Upon studying the current industry, mainstream types of models include:

- **Reasoning Models:** Models such as OpenAI o1 or DeepSeek R1 are optimized for Chain of Thought processing. They excel at complex logical deduction and planning multi-step workflows before taking action. This makes them ideal for the planning phase, where they must decompose difficult user queries into a clear, executable plan before invoking any tools.
- **Function Calling Models** Models such as GPT-4o or Claude 3.5 Sonnet are fine-tuned specifically to output structured data, such as JSON, rather than just conversational text. This capability is efficient for tool configuration and defining output formats. They are critical for the execution phase, as they can reliably select the correct tools and format the arguments precisely to query the database without syntax errors.
- **Multimodal Models** Models such as Gemini 1.5 Pro are trained with multimodal input. These models are essential in an educational content analysis context because they can natively understand both text and visual inputs. This allows the agent to process image frames directly alongside the user's text query without needing a separate description tool to translate visuals into text first.
- **Lightweight Efficiency Models** Smaller models like Llama 3 8B or GPT-4o-mini are faster and cheaper to run. They are perfect for narrow, repetitive tasks such as summarizing retrieved results or classifying user intent, where the deep reasoning capabilities of a larger model are unnecessary. Moreover, the ability to easily train and fine-tune these models facilitates customization in the design of Agentic systems.

Context Engineering Context engineering, represent the memorization capacity, involves crafting the environment in which the LLM operates to ensure accurate and consistent outputs. This field includes several topics that govern how the model interacts with data and users.

- **Prompting Strategies** Prompting is the primary method to instruct the agent, helping it understand the goals and constraints it is dealing with. Techniques such as Query Expansion allow the agent to broaden search terms for better recall. Advanced methods like Chain of Thought (CoT) and Language of Thought (LoT) prompt the agent to articulate its reasoning process step-by-step, ensuring that complex logic is handled systematically before a final answer is generated.
- **State Management** To maintain a coherent interaction, systems must store chat history and the current state. This allows the agent to understand follow-up questions, such as "What happened after that?", by recalling the context of previous turns. State management ensures the agent knows exactly where it is in a multi-step retrieval process, preventing redundant actions.
- **Retrieval-Augmented Generation (RAG)** Agents cannot remember all the information contained in massive educational repositories, and this data is often dynamic. Retrieval-Augmented Generation (RAG) is used to retrieve external information that is not stored within the LLM's internal weights. RAG ensures the agent gets the needed information only when requested, querying the vector database to fetch relevant lecture segments or metadata to ground its answers in factual evidence.

Tool Integration Represent the action capacity, tool integration is the process of connecting external resources (systems, software, hardware) or APIs to the agent, allowing it to perform specific functions that are beyond the capabilities of the LLM alone. This is critical for a automated system, as it enables the agent to interact and manipulate data in real-time. This helps agent truly integrate with the external world, understand its mission from practice instead of static texts, while also pose significant threats of misuse, when agent cause real life mistake and fail to judge. The key to effective tool integration is ensuring that the agent can seamlessly call these tools with the correct parameters and interpret their outputs correctly to inform its next steps.

2.1.3.2 Agent Frameworks

The concept of an Agentic Framework arises from a practical necessity: building reliable autonomous systems requires more than just smart models; it requires a strong structure. Retraining Large Language Models to handle every specific rule or task is extremely expensive and inefficient. Therefore, the industry has shifted towards Agent Frameworks—software architectures designed to control and improve the behavior of AI without changing the model itself.

These frameworks act as a bridge, connecting the creative nature of AI with the reliable logic of software architecture. The Agents are far more than LLMs with tools and context injection; they are intelligent systems with LLMs as the neural core and architecture following software logic. Modern frameworks leverage three main aspects in Agentic design and orchestration:

Multi-agent Orchestration In complex operations, giving a single agent too much information and too many different tasks often causes errors. A single agent has a limited attention span and context window. Frameworks solve this by using Multi-agent Systems. By breaking a large job into smaller parts handled by different agents, developers can assign specific agents to each task—one for visual analysis, one for audio processing, and one for synthesis—ensuring each agent stays focused and accurate.

Agent Memorization Intelligent systems need to remember information over time, but standard LLM treat every question as a new conversation (stateless). Frameworks solve this by building efficient memory systems with hierarchy, often using classes like Session or Memory. These classes act as data handlers containing multiple attributes and methods to help Agents interact with memory to retrieve previous context during run-time. Memory management almost always follows these paradigms:

- **Structured Memory:** Information is organized in a structured way, allowing the system to access relevant context precisely without overwhelming the agent with irrelevant details.
- **Context Window Optimization:** Frameworks implement strategies to optimize the context window. The memory is connected to an external database (usually MySQL) and agentic system get needed context through lazy memory loading of tool results, conversation topics, e.t.c., perform dynamic pruning and memory injection, ensuring that the agent has access to the most relevant information without exceeding its limits, as well as saving computational resources.

Tool Abstraction Frameworks turn standard code functions into Tools that agents can use on their own. The key difference between a normal function and a Tool lies in its data structure:

- **Description:** instructions on what it does and when to use it. The agent read this description when needed, doesn't overflow itself about the internal logic in the orchestration.
- **Input/Output Schema:** a defined structure for the input and output parameters, often in JSON format, that specifies the type and format of data the tool expects and returns. This allows the agent to understand how to call the tool correctly.
- **Logging memory:** functions results, errors logged into memory for agent inference, context injection and debugging.

This allows the framework to give the AI the right capabilities—like a calculator or a database search—allowing the agent to interact with the real world. Good tool design is critical for the performance of the agentic system, as it directly impacts the agent's ability to execute tasks effectively and efficiently. As such, tool design must ensure that the tools are not only functional but also easily accessible and interpretable by the agent, with clear documentation and error handling to facilitate smooth interactions.

2.1.3.3 Langchain and LangGraph

The specific usecase of a our educational system necessitate a flexible, nonlinear, more orchestrated approach. While numerous frameworks exist to harness the capabilities of Large Language Models (LLMs), the intricate requirements of guidance necessitate a high-level, state-aware framework.

Unlike traditional chains or data-centric frameworks like LlamaIndex and SDK, LangGraph is built upon a *State Graph Architecture* where the system is modeled as a directed graph. In this paradigm, each node represents a specialized agent or a discrete computational step, while the edges dictate the transition logic based on the system's current state. This modularity facilitates a sophisticated level of **Localized Context Management**. Instead of burdening a single agent with a big context pile of conversation history, dozen of tools and various educational usecase , the system maintains a shared State object. By granting each agent access only to the specific subset of information required for its immediate task—such as the active knowledge node or the latest student response—LangGraph ensures high-precision reasoning and mitigates the risk of context dilution while maintaining an optimized prompt size.

This granular control over context directly enables Non-linear Branching, allowing the system to mirror the complexity of human instruction. Because the workflow is state-aware, the system can gracefully navigate diverse academic scenarios, utilizing cycles and conditional logic to adapt to the learner's performance. For instance, if a student demonstrates a conceptual misunderstanding, the graph can autonomously trigger a "recovery" branch to revisit foundational prerequisites before returning to the primary roadmap. This deterministic yet flexible orchestration ensures that the AI can handle multifaceted educational use cases without losing track of the student's

long-term learning objectives, a feat that is statistically difficult to achieve with stateless or purely autonomous agent frameworks.

Framework	Core	Characteristic	Strength in Education	Limitation in Education
LlamaIndex	Data-Centric: Optimizing the interface between LLMs and private data.	Retrieval-heavy; focus on indexing and query engines.	Very powerful when it comes to multistep input output manipulation of tools, LLMs and states	Often prioritizes data retrieval over complex, multi-step pedagogical reasoning.
LangGraph	Explicit state management via cyclic graphs. Supporting edges, nodes creation making full control of logic, branching	Stateful, non-linear branching with fine-grained control	Suitable for complex nonlinear task, with stateaware and context, logic localization and independence	Branching acts as a selling point and a potential logical error, when applying complex flow to simple tasks. Most of all, due to its high level abstraction, input output of LLM get unstable
Google SDK	Tool-Centric: Direct, low-latency access to model capabilities.	Functional/Linear; focus on tool-calling and result passing.	Deliver stable tool results, state and worker result if the pipeline is well designed	Becomes unstable and difficult to maintain when including branching workflows.
CrewAI	Role-Centric: Simulating human-like collaborative teams.	Autonomous, task-based collaboration between predefined roles.	High level, well LLM and human interaction facilitation, capable of complex workflow	Can lack the deterministic, full control orchestration required for strict academic knowledge modeling.

Table 2.1: Comparison of LLM Orchestration Frameworks

These survey of various frameworks highlights LangGraph as top framework for educational system. While LlamaIndex offers stable operation with full control data, its data-centered nature often obscures the complex logic and asynchronization required for student-teacher interaction. The Google Generative AI SDK, though efficient for direct tool integration, lacks the complex state-management infrastructure necessary to handle the unstable nature of complex workflow, comparing to predecessor framework. Ability to maintain a persistent state and manage non-linear transitions of LangGraph and CrewAI provides the deterministic backbone required to facilitate a truly adaptive and reliable learning experience.

2.1.4 System Design

This section elucidates the software engineering principles, architectural patterns, and technological foundations utilized to construct the proposed system. Given the integration of high-latency AI inference with real-time educational interactions, the design strictly prioritizes modularity, robust state management, and horizontal scalability. Specifically, it focuses on adapting traditional paradigms to handle the non-deterministic and asynchronous nature of Large Language Model (LLM) orchestration and heavy multimedia processing workflows.

2.1.4.1 Software Paradigm: Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects" which can contain data in the form of fields (attributes) and code in the form of procedures (methods). In this paradigm, computer programs are designed by making them out of objects that interact with one another. Fundamentally, this paradigm relies on four core concepts:

- **Class:** A blueprint or template for creating objects, defining the initial state and behavior.
- **Object:** A specific instance of a class, representing a concrete entity within the system memory.
- **Attribute:** Variables bound to an object that represent its internal state or data.
- **Function (Method):** Procedures associated with a class that define the behaviors or actions an object can perform.

In the context of System Design, OOP is the dominant industry standard because the core components of this architecture are inherently stateful. Unlike simple functional scripts that execute and exit, system components must maintain persistent internal states—including conversation history, active tool configurations. The OOP provides the natural structural to manage this state lifecycle. The system leverages the four main pillars of OOP:

- **Abstraction:** This allows the system to model complex entities by hiding internal implementation details and exposing only relevant operations, allowing the outside workflow logic to interact with high-level methods rather than raw data from lower components.
- **Encapsulation:** Groups related variables and functions into a single unit, protecting the internal state from unintended interference. This ensures that the memory of one component is strictly isolated and cannot be corrupted by the operations of another component running in parallel.
- **Inheritance:** This promotes code reusability by allowing new classes to derive properties from existing ones. We utilize a base class to define shared logic such as error handling, logging, and connection management. Specialized classes then inherit these foundational capabilities while implementing their specific unique behaviors, significantly reducing code duplication.
- **Polymorphism:** This allows objects of different classes to be treated as objects of a common superclass. In the orchestration layer, the system can iterate through a diverse list of components—such as different retrieval engines or analysis modules—and trigger their execution loops using a uniform interface. The orchestrator does not need to know the specific type of component it is invoking, only that every component adheres to the expected interface contract.

2.1.4.2 System Architecture: Microservices

System architecture defines the macroscopic structure of a software system, organizing its major components, inter-process communication, and load distribution. Given the necessity of building an intelligent system responsible for transferring, managing, and indexing massive volumes of educational materials, a monolithic architecture—where the entire application is compiled and deployed as a single unified binary—quickly becomes a massive bottleneck. To mitigate this, the system adopts a **Microservices Architecture**.

Microservices is a design paradigm in which a complex application is decoupled into a suite of small, autonomous, and loosely coupled services. Each service is dedicated to a specific business domain, operates in its own isolated process, and interacts with other services via lightweight protocols (typically REST APIs or message brokers like RabbitMQ/Kafka).

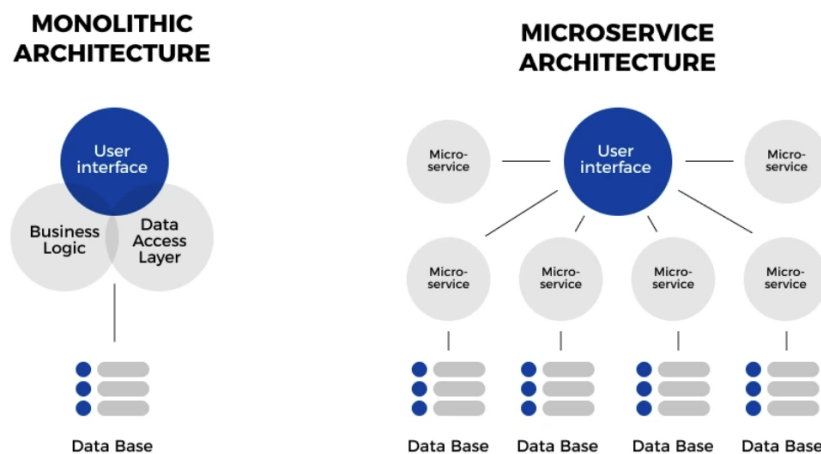


Figure 2.4: Comparison of Microservice and Monolithic architecture

The adoption of Microservices is fundamentally driven by the need to isolate distinct computational workloads. In an AI-driven educational platform, integrating compute-intensive tasks (like LLM inference and embedding generation) with I/O-intensive tasks (like API routing and user authentication) within the same process leads to resource starvation. Decomposing these into microservices yields critical structural advantages:

- **Independent and Asymmetrical Scalability:** This is the most crucial advantage for AI systems. Microservices allow the architecture to scale heterogeneously. Heavy, GPU-bound inference services can be scaled vertically (allocating more VRAM) or horizontally (adding more instances) independently of the lightweight frontend and routing services. This ensures that a spike in users uploading documents does not unnecessarily drain the resources allocated to real-time chat inference, optimizing overall hardware utilization.
- **Fault Isolation and Blast Radius Containment:** In a distributed microservices ecosystem, failures are strictly contained within their respective domains. If the computationally heavy LLM generation service

crashes due to an Out-Of-Memory (OOM) error, the blast radius is limited. The API Gateway, user authentication services, and database layers remain fully operational, ensuring high system availability and allowing users to continue browsing content while the AI service automatically restarts.

- **Technological Flexibility (Polyglot Programming):** Microservices eliminate the constraint of being locked into a single technology stack, allowing engineers to select the optimal tool for each specific task. For instance, AI agents and data processing pipelines are implemented in Python to leverage its rich machine learning ecosystem (PyTorch, LangChain). Simultaneously, the API gateway or real-time notification services might utilize high-concurrency, compiled languages like Go or Node.js. This polyglot approach ensures maximum performance efficiency across the entire application lifecycle.

2.1.5 Technology Stack and Frameworks

The selection of frameworks is driven by the need for high-performance Python execution and seamless integration with AI libraries. These frameworks are chosen to building block for the implementation of the application layer, providing the necessary tools and abstractions to manage the complex interactions between the backend, frontend, and AI components.

2.1.5.1 Backend Development

Backend development serves as the foundational logic of any software application, responsible for data processing, security, and the orchestration of communication between the database and the client. It handles the business logic that users do not see but rely upon for functionality. For an intelligent educational system involving heavy AI workloads, the backend must be capable of handling high-concurrency requests without latency, ensuring that the complex computations of the agents do not block the user experience.

To address these requirements, we utilize FastAPI, a modern web framework for building APIs with Python. While older frameworks like Django were built on synchronous standards known as WSGI, FastAPI adopts the Asynchronous Server Gateway Interface (ASGI). This approach allows the server to handle asynchronous code natively using Python's await syntax. This represents a paradigm shift for AI applications, moving away from blocking, sequential request handling to a concurrent model where I/O-bound tasks—such as waiting for an LLM response or a database query—release the CPU to handle other incoming requests, offering some key features:

- **High Concurrency:** By utilizing the ASGI standard, FastAPI can handle thousands of simultaneous connections (such as multiple users uploading videos or chatting with agents) without the blocking issues inherent in traditional synchronous frameworks.
- **Data Validation:** It integrates tightly with Pydantic, a data validation library. This ensures that the complex JSON outputs generated by AI agents are rigorously validated against strict schemas before being sent to the frontend, preventing runtime type errors.
- **Performance:** Built on top of Starlette, FastAPI offers execution speeds comparable to compiled languages like Go, which is critical for minimizing the latency overhead in the educational content processing pipeline.

2.1.5.2 Frontend Development

Frontend development is critical as it represents the intersection between the system's logic and human interaction. It focuses primarily on user experience, translating raw data into an accessible, responsive interface. The goal is to render the application elegant, fast, and secure, abstracting the complexity of the underlying educational analysis tools from the end-user. With this definition, we selected a library that prioritizes a declarative and component-based approach.

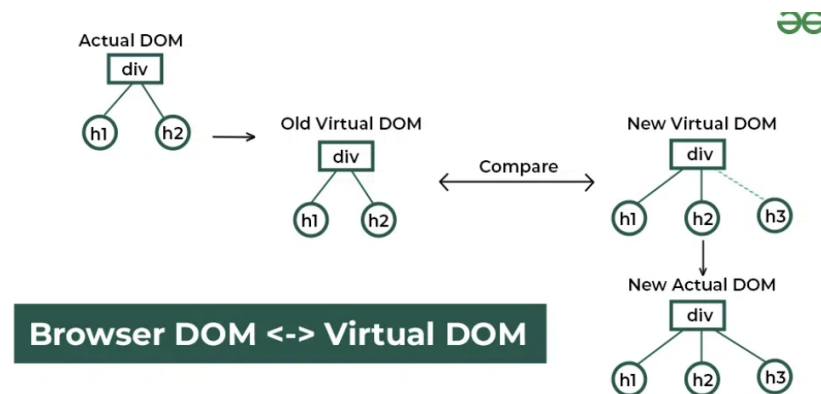


Figure 2.5: React Virtual DOM; Source: GeeksforGeeks

React is a JavaScript library for building user interfaces, created by Meta in 2011 to address the challenges of building large-scale, dynamic web applications. Unlike previous generations of tools that relied on imperative manipulation of the browser's Document Object Model (DOM), React introduced a declarative model utilizing a Virtual DOM. This is a lightweight memory representation of the UI. When the application state changes, React calculates the difference and updates the real DOM efficiently. This approach allows developers to build complex, stateful interfaces that update in real-time without compromising performance, offering some key features:

- **Component-Oriented Architecture:** React encourages breaking the UI into independent, reusable pieces called Components (e.g., a SearchBar, a ContentView). This isolation allows for modular development and easier maintenance of the code.
- **Unidirectional Data Flow:** Data flows in a single direction, from parent to child components. This structure makes the application state predictable and easier to debug, which is essential when managing the streaming data from the AI agents.
- **Rich Ecosystem:** Being a library rather than a rigid framework, React allows for the integration of specialized tools for routing, state management, and visualization, providing the flexibility needed to build a custom dashboard.

2.1.5.3 Ollama: Local Model Inference and Optimization Infrastructure

Ollama is an advanced abstraction layer designed for managing and executing LLMs on local infrastructure. It is built upon the llama.cpp core, providing an efficient environment for model orchestration and hardware acceleration.

Quantization Techniques

A critical feature of local inference is **Quantization**, the process of reducing the precision of model weights from high-bit floating-point formats (e.g., FP16 or BF16) to lower-bit integers (e.g., 4-bit or 8-bit). This technique significantly reduces the VRAM footprint and memory bandwidth requirements:

$$w_q = \text{round}\left(\frac{w}{\Delta}\right) \quad (2.4)$$

where w is the original weight and Δ is the scaling factor. Quantization enables large-scale models to run on consumer-grade hardware with minimal degradation in perplexity and reasoning capabilities.

GGUF Format and Resource Management

Ollama utilizes the **GGUF (GPT-Generated Unified Format)**, a binary format designed for rapid model loading and efficient metadata storage. Key technical characteristics include:

- **mmap Compatibility:** Allows models to be mapped directly into memory, enabling nearly instantaneous startup times.
- **Unified Execution:** Supports flexible memory offloading, where layers of the model can be distributed between the GPU (VRAM) and CPU (System RAM) based on available resources.
- **Modelfile Abstraction:** Provides a structured way to define system prompts, temperature settings, and context window sizes within a single configuration file, ensuring deterministic behavior across different local environments.

2.1.5.4 The Langchain Ecosystem

As mentioned in Subsection 2.1.3.3, Langchain is a powerful framework for our educational system, with State Graph Architecture. Moreover, it is one of the first and most popular agentic frameworks in the industry, with a large community and rich documentation. The accumulated contributions to the ecosystem and agentic research using Langchain have provided us with a wealth of tools and integrations:

- **Pre-built Agents:** Langchain offers a variety of pre-built agents that can be easily customized for specific use cases, such as question-answering, summarization, and multi-step reasoning. In 2026, it publishes Deep-Agent, capable of extremely long term reasoning and complex workflow orchestration, and currently leads the harnessing researching trend in the industry.
- **Langfuse:** Langfuse, among with Langsmith, is one of the most powerful tools for agentic system development. It provides a comprehensive observability platform for monitoring and debugging agent interactions, allowing developers to trace the decision-making process of agents in real-time. This is critical for diagnosing issues in complex workflows and ensuring that the system behaves as expected, making agentic operation more transparent, manageable and less errorprone.
- **Context Management:** Langchain includes built-in context management capabilities, enabling agents to maintain context across interactions and manage state effectively. Those includes utilities for memory management (adapting to multiple external databases), conversation history, and dynamic context injection, which are essential for building complex agentic system for long interactions over time.

2.1.6 Database System Design & Management

Database is where you store necessary information where application can read, write, update and delete via query or integrated calls or methods. In the domain of modern software engineering, data handling has become increasingly complex. To address this, robust architectures utilize a Polyglot Persistence strategy, utilizing distributed storage technologies optimized for different data modalities.

Table 2.2: *Polyglot Persistence Architecture: Strategic allocation of storage engines.*

Storage Engine	Data Modality	System Role	Theoretical Justification
Relational (MySQL)	Structured Data	User Authentication, Billing, Access Logs	Enforces ACID properties for transactional integrity and supports complex relational joins for administrative analytics.
NoSQL (MongoDB)	Semi-Structured Data	Semantic Metadata, Analysis Tags, JSON Outputs	Provides schema polymorphism to handle variable outputs from different AI agents (e.g., OCR vs. Object Detection) without sparse tables.
Object Storage (MinIO)	Unstructured Binary	Lecture Materials, Educational Assets, Audio Clips	Decouples storage from compute to prevent database bloat; utilizes Erasure Coding for high-throughput parallel streaming during ingestion.
Vector Database (Milvus)	High-Dimensional Vectors	Embeddings Index, Semantic Search	Optimized for Approximate Nearest Neighbor (ANN) algorithms (e.g., HNSW), enabling efficient similarity search in latent spaces where traditional B-Tree indexing fails.
Graph Database (Neo4j)	Multi-hop Graph Data	Knowledge Graph, Prerequisite Relationships	Optimized for traversing complex relationship networks; enables recursive queries and multi-hop reasoning to connect disparate educational concepts.

2.1.6.1 Relational Database (RDBMS)

Relational Databases are mathematically founded on Set Theory and First-Order Predicate Logic. They organize data into tuples (rows) and attributes (columns) within strictly defined relations (tables). The defining characteristic of this model is the enforcement of ACID properties (Atomicity, Consistency, Isolation, Durability), which guarantees that database transactions are processed reliably.

RDBMS is typically utilized strictly for data that requires high integrity and deterministic relationships, such as User Management and Billing. It acts as the source of truth for entity relationships, enabling complex SQL JOIN operations (e.g., selecting all learning materials uploaded by a specific role within a timeframe).

The adoption of RDBMS for core administrative data offers specific advantages to system stability.

MySQL

MySQL is utilized as the primary Relational Database Management System. It handles core entities such as user accounts, roles, and administrative logs, ensuring that all structured interactions follow strict relational integrity.

- **Transactional Integrity:** In a multi-user environment where credits or permissions are updated frequently, RDBMS ensures that race conditions do not occur. For instance, if a user purchases a subscription while simultaneously uploading an educational asset, the ACID properties ensure the account balance is updated accurately before the upload permission is granted.
- **Data Normalization:** By structuring data into normalized tables (e.g., Users, Roles, Logs), redundancy is minimized. This ensures that a change in a user's details is reflected system-wide instantly, without needing to update thousands of duplicate records.
- **Complex Querying:** The robust SQL engine allows administrators to perform audit trails and analytics efficiently, such as tracking system usage patterns across different user groups, which is computationally expensive in non-relational systems.

2.1.6.2 Non-Relational Database (NoSQL)

To address the limitations of rigid RDBMS schemas, Non-Relational databases emerged, often adhering to the CAP Theorem (Consistency, Availability, Partition Tolerance). Unlike RDBMS which prioritizes strong Consistency, NoSQL databases like MongoDB often prioritize Partition Tolerance and Availability, offering a flexible schema-less design.

Document stores are typically the primary engine for storing the Semantic Metadata derived from educational content analysis. Metadata derived from curriculum analysis is highly variable; one resource might contain OCR text, while another contains only Audio Transcripts or Face Recognition data. Document stores handle these as dynamic BSON (Binary JSON) documents, allowing fields to be added on the fly without the sparse matrices typical of rigid SQL tables.

The utilization of a Document Store is critical for the AI-driven nature of context retrieval:

MongoDB

MongoDB serves as the high-performance document store for managing the semi-structured metadata generated by AI agents. It allows the system to store diverse analysis results—such as transcripts, OCR results, and agentic reasoning logs—within a single, flexible schema.

- **Schema Agnosticism:** AI agents are constantly evolving. If an "Object Detection" agent is upgraded to output new attributes (e.g., "Confidence Score" or "Bounding Box Coordinates"), Document stores allow the new data to be saved immediately without migrating the entire database schema or breaking existing application logic.
- **Read Performance for Aggregates:** In a Knowledge-QA scenario, the system needs to retrieve the Video Title, Description, Transcripts, and Vector IDs simultaneously. In an RDBMS, this might require joining multiple tables. In a NoSQL system, this data can be stored as a single nested document, allowing the application to retrieve the entire context of a lesson in a single, high-speed read operation.
- **Horizontal Scalability:** As archives grow to thousands of hours of lecture content, metadata storage requirements increase. Native sharding capability allows data to be distributed across multiple servers economically, ensuring lookup times remain low even as the dataset grows.

2.1.6.3 High-Performance Object Storage (MinIO)

While RDBMS and NoSQL excel at storing alphanumeric data, they are fundamentally inefficient for storing massive binary large objects (BLOBs) like lecture recordings, leading to database bloat and performance degradation. Object Storage systems like MinIO treat data not as a file hierarchy or a table, but as discrete objects stored in a flat address space.

High-performance architectures utilize Object Storage to handle the High Volume nature of raw educational media. It utilizes Erasure Coding—a mathematical algorithm that splits data into fragments and expands them with redundant data pieces—allowing for data recovery even if multiple drives fail. This approach decouples storage from compute; the heavy educational artifacts are offloaded to Object Storage, while the databases store only the lightweight reference pointers (URLs) to these objects.

Offloading binary data to Object Storage provides the infrastructure necessary for high-throughput educational media processing:

MinIO

MinIO is deployed as a high-performance, S3-compatible object storage server. It is responsible for hosting all raw multimedia assets, including lecture videos, extracted images, and audio segments used for AI training and inference.

- **Decoupling Storage and Compute:** By separating the raw files from the metadata databases, "database bloat" is prevented. This ensures that the RDBMS remains lightweight and responsive for user queries, while the Object Store handles the heavy I/O load of content streaming.
- **Parallel Streaming for AI Ingestion:** When an Agent analyzes a lecture recording, it requires high-speed access to the file to extract frames. Object Storage supports parallelized byte-range requests, allowing the AI to read multiple parts of a media file simultaneously. This significantly reduces the time required for the ingestion and preprocessing phase compared to reading from a standard file server.
- **Cost-Efficiency and Redundancy:** Erasure Coding allows the system to survive hardware failures without data loss, while consuming less storage space than traditional replication methods. This is vital for maintaining a massive archive of media files economically.

2.1.6.4 Vector Database

Traditional databases are designed for structured data and are inefficient for handling high-dimensional vector data generated by AI models. Vector databases are optimized for storing and querying high-dimensional vectors, which represent the semantic content of lecture segments. They utilize Approximate Nearest Neighbor (ANN) algorithms, such as Hierarchical Navigable Small World (HNSW) graphs, to enable efficient similarity search in latent spaces where traditional indexing methods fail.

The Vector Database is the backbone of the Retrieval-Augmented Generation (RAG) paradigm, allowing the system to perform semantic search based on the embeddings generated by the AI agents. When a user query is processed, it is transformed into an embedding vector, which is then compared against the vectors stored in the database to retrieve the most semantically relevant knowledge segments.

The adoption of a Vector Database is critical for enabling the advanced retrieval capabilities required for educational analysis:

Milvus

Milvus is the chosen vector database engine for SmartEdu. It indexes high-dimensional embeddings generated from course documents and lecture transcripts, providing sub-second retrieval times for semantic similarity queries across millions of vectors.

- **Efficient Similarity Search:** Traditional databases cannot efficiently query high-dimensional vectors. Vector databases provide specialized indexing structures to handle this efficiently.
- **Scalability for Large Datasets:** As the number of educational segments grows, the vector database can scale horizontally to maintain low latency for similarity searches, ensuring that retrieval times remain fast even as the dataset expands.

- **Integration with AI Workflows:** Vector databases are designed to integrate seamlessly with AI pipelines, supporting real-time insertion and retrieval of embeddings.
- **Support for Dynamic Data:** As new educational materials are ingested and analyzed, their embeddings can be added to the vector database in real-time, allowing the system to continuously evolve and improve its retrieval capabilities without downtime.

2.1.6.5 Graph Database

While vector databases handle semantic similarity, the structural relationships between educational concepts—such as prerequisites and curriculum hierarchies—are best modeled as a graph.

Neo4j

Neo4j is utilized to store the "Backbone Graph" of the knowledge base. It represents educational entities (concepts, lessons, modules) as nodes and their pedagogical relationships as directed edges.

- **Multi-hop Reasoning:** Unlike relational databases that require expensive JOIN operations, Neo4j allows for efficient traversal of deep relationship chains. This is critical for detecting knowledge gaps that may span multiple prerequisite levels.
- **Cypher Query Language:** The use of Cypher enables the system to express complex pedagogical patterns, such as "Find all prerequisite concepts of concept X that the student has not yet mastered," in a readable and performant manner.
- **Dynamic Schema:** The graph model allows the knowledge structure to evolve as new educational materials are added, without needing to redefine rigid table structures.

2.1.6.6 Database Management and Observation

Managing multiple heterogeneous database engines requires a unified observation layer to ensure data consistency and facilitate debugging during development.

DbGate

DbGate is an open-source, multi-database manager used as the primary dashboard for the SmartEdu infrastructure. It provides a unified web interface to inspect and manipulate data across MySQL and MongoDB instances simultaneously.

- **Unified Interface:** Reduces the overhead of switching between multiple specialized management tools.
- **Schema Visualizer:** Helps developers understand the evolving structure of NoSQL documents and Relational tables in real-time.
- **Query Benchmarking:** Allows for testing and optimizing complex queries before they are integrated into the application's repository layer.

2.1.7 Deployment and Containerization

Deployment is the process of making a software application available for real applicational usage. This section discusses the deployment strategy and the use of containerization to ensure that the system can be reliably and efficiently deployed for our AI pipeline and highperformance stacks as discussed above.

2.1.7.1 Docker

Docker [4] is a containerization platform that enables developers to bundle applications and their required components into isolated units called containers. Each container acts as a lightweight, virtualized environment that includes all the necessary elements for the application to run. Because containers are portable and consistent, they work reliably across various environments such as on-premise systems, cloud platforms, and clustered infrastructures.

- **Portability:** A major advantage of Docker is its strong portability. Containers package the application together with all its dependencies, making it simple to move workloads between different environments without compatibility issues. Whether running on a local machine or a cloud server, a Docker container behaves the same way, ensuring consistency throughout the development and deployment pipeline.

- **Isolation:** Docker provides strong isolation between applications and their dependencies. Each container operates independently, preventing interference between applications running on the same host. This isolation enhances both security and stability, especially in shared hosting environments.
- **Resource Efficiency:** Docker containers are more resource-efficient than traditional virtual machines. Since containers share the same operating system kernel, they consume far less memory and disk space. This allows more applications to run on a single machine, reducing hardware overhead and simplifying resource allocation.
- **Scalability:** Docker supports efficient scaling of applications. By using orchestration tools such as Kubernetes or Docker Swarm, developers can easily launch multiple container instances and distribute workloads as needed. This flexibility allows systems to rapidly adjust capacity to meet varying levels of demand.
- **Faster Development and Deployment:** Docker improves the speed and efficiency of both development and deployment processes. Developers can work in isolated, reproducible environments identical to production, minimizing integration issues. The deployment process becomes more automated, streamlined, and less prone to errors, saving significant time.

Docker has transformed modern software development by simplifying the creation, deployment, and management of services. Its emphasis on portability, isolation, resource efficiency, scalability, and rapid deployment makes it an indispensable tool for both development and operations teams.

2.1.7.2 AWS

AWS (Amazon Web Services) [14] is a comprehensive cloud computing platform that provides a wide range of services, including computing power, storage, and databases, all needed for the containerized environment to actually run. Furthermore, for that running containerized applications to be publicly accessible, it needs to be hosted on a cloud platform that expose accessible domain to the internet. AWS provides that, along with various tools and services for managing and orchestrating containerized applications, and since its foundation it has been developing itself to be the infrastructure that brings:

- **Scalability:** AWS offers a wide range of services that can be easily scaled up or down based on demand. This allows businesses to handle varying workloads without worrying about infrastructure limitations.
- **User-Friendly Interface:** AWS provides a user-friendly web interface and command-line tools that make it easy for developers to manage their resources and deploy applications. This accessibility has contributed to its widespread adoption.
- **Flexibility:** AWS supports a wide variety of programming languages, frameworks, and operating systems, allowing developers to choose the tools that best fit their needs. This flexibility has made it a popular choice for a diverse range of applications.
- **Global Reach:** With data centers located around the world, AWS allows businesses to deploy applications closer to their users, reducing latency and improving performance. This global infrastructure is a significant advantage for companies with an international user base.
- **Security:** AWS provides robust security features, including encryption, identity and access management, and compliance certifications. This focus on security has made it a trusted platform for businesses of all sizes, including those in regulated
- **Reliability:** Among the most wellknown cloud providers such as Google, AWS have a strong reputation for reliability, with such few records of failures (esspecially comparing to Google, from my own experience). AWS has a strong track record of uptime and reliability, with multiple availability zones and data centers designed to ensure high availability. This reliability is crucial for businesses that require consistent access to their applications and data.

AWS has become a dominant player in the cloud computing market due to its comprehensive service offerings, scalability, user-friendly interface, flexibility, global reach, security features, and reliability. It continues to innovate and expand its services, making it a preferred choice for businesses of all sizes looking to leverage cloud technology for their applications and infrastructure needs. Finally, with a big community and ecosystem, AWS provides extensive documentation, support, and third-party integrations, making it easier for newbies to find resources and solutions for their specific use cases. It is a good start to quickly deploy a reliable scalable system without worrying about the underlying infrastructure, allowing the research to concentrate on the core AI and educational system development.

2.1.8 Security: OAuth 2.0 Authorization

OAuth is a widely used authentication and authorization protocol in backend systems across various programming languages and frameworks. Here are some general benefits of incorporating OAuth into the backend system:

- **Security:** OAuth provides a secure way for users to grant third-party applications limited access to their resources without sharing their credentials. This reduces the risk of unauthorized access and protects user data.
- **User Experience:** OAuth enables seamless authentication and authorization experiences for users by allowing them to log in to your application using their existing accounts from trusted providers. This eliminates the need for users to create and manage yet another set of credentials.
- **Single Sign-On (SSO):** OAuth facilitates single sign-on capabilities, enabling users to access multiple applications and services with a single set of credentials. This improves user convenience and reduces the likelihood of password fatigue.
- **Granular Permissions:** OAuth allows you to define and enforce granular permissions for accessing resources. This ensures that third-party applications only have access to the specific resources they need, enhancing security and privacy.
- **Scalability:** By offloading user authentication and authorization to OAuth providers, your backend system can better handle scalability challenges. OAuth providers are equipped to handle large volumes of authentication requests efficiently, allowing your application to scale more effectively.
- **Standardization:** OAuth is a widely adopted and standardized protocol, supported by major technology companies and libraries across different programming languages. Leveraging OAuth in your backend system ensures compatibility with a broad ecosystem of tools and services.
- **Compliance:** OAuth implementations often come with built-in support for compliance with regulatory requirements such as GDPR (General Data Protection Regulation) or HIPAA (Health Insurance Portability and Accountability Act). This can simplify your compliance efforts and reduce legal risks.
- **Developer Ecosystem:** OAuth has a large and active developer community, providing extensive documentation, libraries, and support for various programming languages and frameworks, such as PyJWT for protocol schema, etc. This makes it easier for developers to implement OAuth in their backend systems and troubleshoot any issues that may arise.

2.2 Related Work

2.2.1 Document-Knowledge Modeling

Converting documents into structured knowledge representations has become a critical task for so long in the industry and education specifically. To replace manual curation in educational contexts, which is time-consuming and not scalable, prior attempts have been focusing on the power of entity extraction, prerequisite relation reasoning, and RAG. This has been indeed a fine strategy, given the success of early supervised models like PREREQ [7] using latent topic representations with Siamese networks to infer course dependencies, which achieved over 70 % Precision@100 scores on the University Course and NPTEL MOOC datasets. However, the traditional methods of entity extraction and relation reasoning often rely on rule-based systems or pure statistical machine learning techniques, which can struggle to capture the complexity and nuance of educational content's relationships. Moreover, recent researches [19, 9] have proven that models fail drastically to access relevant information when documents exceed a certain length. That is the reason why though prior researches gained impressive results on medium retrieval benchmarks, they are not easily applied in industry when it comes to massive and unstructured enterprise data.

To address this, LLM-based frameworks have been widely proposed. A prime example is KGGen [11], an automated system that utilizes a clustering algorithm to group synonymous entities and relations, resulting in denser and better-connected graphs that scored an impressive 81.03% average fact accuracy on the MINE-2 (Measure of Information in Nodes and Edges) benchmark. The EDC [20] framework extends this capability by extracting, automatically defining schemas, and canonicalizing knowledge triples without predefined schemas, consistently surpassing state-of-the-art baselines on complex datasets like WebNLG, REBEL, and Wiki-NRE.

Figure 2.6: Architecture of the EDC framework for extracting and canonicalizing knowledge triples.

However, LLMs have deadly drawbacks. First and most popular of all is the “hallucinations problem” leading to inaccurate, destructured extractions. To mitigate this, GraphJudge [5] proposes using a fine-tuned open-source LLM as a “judge” to score and filter out incorrect knowledge triples generated by naive LLMs. Validated on general (REBEL-Sub, GenWiki) and domain-specific benchmarks (SCIERC, Re-DocRED), it achieved over 90% judgement accuracy. Furthermore, for large-scale educational and enterprise systems requiring cost and latency optimization, comprehensive hybrid pipelines have proven highly effective in practices. For instance, EduKGPipeline [1] establishes an end-to-end framework fusing traditional text extraction with transformer-based weighting, demonstrating high precision on educational transcripts from the CCI dataset.

Figure 2.7: *The end-to-end EduKGPipeline framework for educational knowledge graph construction.*

Similarly, recent efficient GraphRAG architectures [10] substitute expensive LLM extractions with classical dependency parsing to maintain scalability, retaining 94% of LLM-based performance on RAGAS metrics when evaluated on real-world enterprise datasets like CCM (Custom Code Migration). Finally, CoDe-KG [2] utilizes syntactic decomposition to process complex sentences, thereby significantly enhancing information recall by an 8-point gain on rare relations across the REBEL and WebNLG2 benchmarks.

Another inevitable error lies in the LLMs’ limited context length. It is good when discovering short-dependent context/entities/relationships in small sections but fails for long dependencies of extremely long documents and course information. Trying to inject an excessive amount of information even when cut by windows, until now with the current level, only leads to redundant costs from multiple calls, information overload, and decreased accuracy. To uncover hidden relationships and deduce learning paths beyond textual proximity, studies have applied the iterative iPRL [13] method combined with graph-ranking algorithms to learn prerequisite relations without massive labeled data, significantly outperforming baselines by up to 18% in average F1 scores on multi-domain Textbook and MOOC datasets. Notably, as graphs grow, analyzing their macroscopic structure becomes crucial. The Dynamic Knowledge Graph (DKG) [3] approach divides KGs into discrete time snapshots and applies network science to track knowledge evolution. To recommend optimal learning paths, DKG introduces the KC2 (Knowledge Connector Candidate) algorithm, which uses closeness centrality to suggest connections between disconnected knowledge islands. Additionally, to cluster knowledge into logical chapters or modules, the Leiden [16] algorithm is utilized to optimize the CPM quality function, ensuring that knowledge communities are absolutely well-connected and not disjointed—a common flaw of the traditional Louvain algorithm.

Figure 2.8: *Dynamic Knowledge Graph (DKG) approach tracking knowledge evolution over time.*

2.2.2 Generative AI and Agentic Systems in Education

Bridging the gap between raw document extraction algorithms and end-user educational platforms is the integration of Generative AI and Large Language Models (LLMs). The increasing accessibility and reasoning capabilities of Generative AI tools have led to widespread exploration in educational settings, fundamentally transforming traditional teaching and learning paradigms [?]. Unlike classical Intelligent Tutoring Systems (ITS) that rely on static, rule-based logic, modern LLM agents are capable of autonomous reasoning, tool manipulation, and orchestrating complex pedagogical tasks.

2.2.2.1 Advances in Educational Agents

Recent surveys highlight the versatility of LLM agents in automating multifaceted educational workflows, ranging from curriculum design to generating highly contextualized feedback [?]. These agents operate dynamically, leveraging multi-hop reasoning and continuous interaction to adapt to a student’s cognitive state. A qualitative study involving educators demonstrated that the core impact of generative models spans three critical pillars: active teaching and learning, administrative automation, and personalized assessments [?].

A prominent, large-scale empirical example of this shift is the integration of AI within Harvard University’s CS50 course. By deploying a suite of specialized AI-based software, the course successfully approximated a 1:1 teacher-to-student ratio. Crucially, the system was designed with pedagogical boundaries—guiding students toward conceptual understanding rather than merely offering outright code solutions. This constrained autonomy proved highly effective in code explanation, style improvement, and administrative querying, demonstrating the practical viability of AI as a ubiquitous teaching assistant [?].

2.2.2.2 Challenges and Deployment Bottlenecks

Despite the profound opportunities, deploying LLM agents in educational ecosystems introduces substantial systemic and ethical challenges. The inherent probabilistic nature of LLMs frequently leads to “hallucinations,”

where the model confidently generates factually incorrect or pedagogically flawed information. Furthermore, there is a significant risk of cognitive offloading, where students develop an overreliance on AI, thereby degrading their independent problem-solving skills [?].

From a practitioner's perspective, integrating these models into open-source or institutional infrastructure often encounters severe operational bottlenecks. Empirical analyses of LLM-based open-source projects categorize these challenges primarily into model configuration issues, connection instabilities, and feature misalignment, all of which hinder seamless deployment [?]. Furthermore, theoretical analyses of LLM representations reveal phenomena such as "semantic collapse," where the continuous manifold of the model's activations struggles to maintain stable, logically sound ontologies over continuous, long-term reasoning tasks without rigid architectural constraints [?]. These limitations emphasize the need for robust orchestration frameworks to tightly control the semantic boundaries of AI agents in an academic context.

2.2.2.3 Verifiable Evaluation of Agentic Workflows

As educational AI transitions from static next-token prediction to autonomous multi-agent collaboration, ensuring the reliability, alignment, and transparency of these systems becomes a critical engineering challenge. Traditional benchmarks, which typically evaluate single-response accuracy, are inadequate for assessing the non-linear, multi-step nature of agentic workflows.

This necessitates the development of rigorous, process-aware evaluation frameworks. Architectures like the Adaptive Evaluation Multi-Agent (AEMA) framework have been proposed to address this gap. AEMA provides an auditable, multi-step evaluation mechanism that operates under human oversight, ensuring that the agents' internal decision-making processes, tool selections, and collaborative strategies are verifiable and transparent [?]. Implementing such verifiable frameworks is essential to mitigate the risks of unconstrained autonomy, ensuring that the integration of AI into education remains both trustworthy and pedagogically effective.

2.2.3 Educational Intelligent System

Smart educational systems increasingly leverage AI and educational KGs to personalize learning. This evolution is evident in both academic research and commercial platforms.

In academia, [21] (along with variants like K12EduKG and MOOC-KG) has been developed to uniformly model knowledge from textbooks and online courses. The [1] framework automates KG construction from unstructured PDF documents by extracting keywords via Keybert and performing semantic linking with LLMs. Furthermore, MEduKG pushes beyond text limitations by integrating multi-modal data into the graph, thereby enhancing the system's comprehensiveness and vividness for students.

Researches relating to this field has also reached the market. Popular platforms worldwide like Khanmigo leverage Generative LLMs to provide personalized tutoring and real-time feedback, with enormous available resources. For Vietnamese representative, VioEdu utilizes a manually curated knowledge tree to structure its curriculum, while EdMicro, specialized for enterprises, employs a skill matrix to map out learning objectives and prerequisites. These systems represent different approaches to integrating AI into education, each with its own strengths and limitations in terms of scalability, personalization, and content structuring. Based on practical surveys and evaluation criteria, current educational systems in the market exhibit various characteristics regarding AI and knowledge structuring. A comparative analysis of these platforms is presented in Table 2.3.

Criteria	EduKG	Khan Academy	Docebo	VioEdu (FPT)	EdMicro
Adaptive learning: Personalize the learning process based on needs, abilities, and preferences.	High	High	Low	Medium	Medium
Intelligent Recommendations: Some personalized suggestions courses, videos based on preferences, and areas of improvement.	High	High		Low	
Natural language processing for queries: Based on the questions from learners, it can give out the answer based on the content the LMS has in its DATABASE.	High	Medium			
Learning analytics/Progress Tracking: AI-based analytics track and analyze user progress. Can visualize using tracking board, and leaderboard.	Medium	High	High	High	High
Text-to-audio content: Provide audios for text content.		High			
Interactive learning method: Interactive content, such as quizzes, coding exercises, and simulations, can engage learners and reinforce learning.	Low	Low	Medium	Medium	Medium
Dynamic KG Reasoning & Roadmap: Automatically extract concepts and mathematically reason long-term prerequisite learning paths without manual expert input.					

Table 2.3: Comparative analysis of representative educational intelligent systems based on key AI and knowledge features.

2.2.4 Conclusion and Research Gaps

During the survey of mentioned representative researches, upon studying from multiple peer reviews and self-reviews of those, this report pinpoints the following core research gaps in the current landscape:

Limitations in Automation and Cost Optimization: All mentioned big systems heavily depend on manually built knowledge trees or skill matrices. Ordered course contents but lack inner modelling makes the system less adaptive to students' needs and preferences, leading to good courses recommendations but terrible inner course concept recommendations. This makes learning path become a linear process and not flexible on the micro scale, with little to no revision and update, which is not ideal for the dynamic nature of knowledge and students' needs. The lack of automation also limits the scalability of these systems, as manual curation becomes increasingly impractical as the amount of content grows.

Lack of Connectivity and Transparency in Lesson Structures: Current module clustering methods utilized in text-mining often produce disconnected or fragmented knowledge groups. Existing learning platforms typically lack a standard, mathematically proven network algorithm for dividing study modules, leading to disjointed curricula that disrupt the learners' cognitive flow.

Static Roadmap Reasoning and Gap Detection: While current LMSs and AI tutors provide generated advice, they are actually still relied on traditional long form docs, lacking the network structure and mathematical, statistical calculations to map out prerequisites dynamically. The identification of critical "knowledge gaps" remains static and reactive, failing to automatically trace back to the fundamental bridge concepts (based on network centrality) that cause student bottlenecks.

So far [1] and its lineages have made a significant attempt to propose solutions to those mentioned problems with an end-to-end pipeline that fuses traditional text extraction with transformer-based weighting. [3] has proposed what [1] is missing: a mathematical efficient community detection method, furthermore solve the connectivity and transparency issues in lesson structures that almost all of previous pipeline encountered. However both have their own limitations. The former is still dependent on LLMs, which causes knowledge fragmentation, while the latter pipeline is not fully completed, good for knowledge modeling and analysis but not inference and recommendation. Moreover, both are not yet widely applied due to the lack of a comprehensive system that make use of their graph

Chapter 3

Requirements Specifications

The limitations discussed in Subsection 2.2.4 have shaped the technical vision for the proposed system. To address these gaps and deliver a robust, scalable educational platform following our goal (Section 1.2), this section outlines the explicit Functional and Non-Functional Requirements, as well as visualizing use—case design. These requirements are derived directly from the systemic constraints and the pedagogical needs of learners for a more personalized, diversified and comprehensive learning assistance system.

3.1 Functional Requirements

3.1.1 Knowledge Modelling and Management

As a key objective feature, knowledge modelling construct knowledge base, foundation to all other key features. This functional category encompasses the capabilities required to process raw educational inputs into a structured knowledge base.

- **Data Ingestion and Parsing:**

- The system must be able to ingest unstructured, text-dense academic formats (e.g., PDF textbooks, syllabus guidelines).
- The system must accurately parse text into discrete chunks while retaining crucial positional metadata (e.g., source file origin, exact page numbers).

- **Semantic Extraction and Normalization:**

- The system must extract fundamental educational concepts and their explicit pedagogical relationships from the ingested text.
- The system must normalize extracted entities to a canonical form to ensure conceptual consistency and prevent any form of semantic fragmentation (even of different course) across the knowledge base.

- **Hierarchical and Relational Organization:**

- The system must organize extracted entities into a standardized hierarchical topology.
- The system must automatically group highly related concepts into cohesive educational clusters or chapters.
- The system must explicitly define and store directional "prerequisite" relationships between concepts to enable automated learning path generation.

- **Storage Capability:**

- The persistence mechanism must guarantee a database structure that facilitates high-speed, complex relational querying and subgraph retrieval during live interactions.
- To ensure diversity of learning input, storage design must ensure efficient storing of enormous data with reasonable latency.

3.1.2 Intelligent Teaching Assistance

To utilize modeled knowledge base as well as guarantee satisfactory learning experience and result, the design of agentic system that serves as our Teaching Assistance is crucial. This category defines the required behaviors and reasoning capabilities of the automated teaching assistant.

- **Intent Recognition and Context Orchestration:**

- The system must classify user intents (e.g., Retrieve information, Generate roadmap, Clarify concepts) and autonomously route the request to the appropriate internal workflow.
- The system must dynamically gather necessary context before responding, which includes retrieving the student's personal mastery history and the relevant subset of the educational knowledge base.

- **Intelligent Reasoning and Supervision:**

- The system must support a Socratic teaching interaction; it must be capable of generating probing questions to assess student comprehension rather than merely providing direct answers.
- The system must objectively evaluate the student's response to determine whether to authorize progression to the next prerequisite topic.
- The system must be able to autonomously detect when it lacks sufficient context to answer a question and execute a fallback strategy that synthesizes information from the broader knowledge base.

- **Response Verifiability and Fallback:**

- To guarantee grounded responses, the system must prioritize explicit document referencing, citing exact source pages for its generated answers.
- The system must execute a broader knowledge synthesis fallback only when direct contextual concepts are missing from the immediate document.

3.1.3 Student Interaction and System Navigation

This category defines the functional requirements concerning the end-user's experience, accessibility, and direct interaction with the intelligent system.

- **Identity and Access Security:**

- The system must provide secure authentication capabilities (Login/Logout) to uniquely identify users.
- The system must maintain isolated, tamper-proof session states for each user to securely record individual learning progress without cross-user data exposure.

- **Interactive Dialogue and Visualization:**

- The system must provide a natural language chat interface that supports continuous, multi-turn conversational interactions and real-time streaming responses.
- The system must dynamically render visual learning roadmaps, empowering students to make informed, user-centric choices regarding their educational progression rather than following a static curriculum.
- The interface must consolidate essential learning tools, providing seamlessly integrated functional components such as conversational history logs, a synchronized document viewer, and progress summarization dashboards.

- **Advanced Navigation and Content Discovery:**

- The system must implement an intuitive, multi-level navigation structure (e.g., breadcrumb trails) to help students orient themselves within complex knowledge hierarchies.
- The system must incorporate a powerful semantic search utility, allowing users to rapidly query and locate specific educational materials, graph concepts, or historical interactions.

- **Contextual and Responsive Feedback:**

- The system must be capable of emitting explicit interface triggers alongside its textual responses to automatically synchronize the user's interface (e.g., auto-scrolling the document viewer to the exact referenced page).
- Notifications and intelligent pedagogical prompts must be delivered in a non-intrusive, accessible manner that captures user attention while preserving the fluidity of the learning experience.

3.2 Non-Functional Requirements

To create a powerful, reliable, and user-friendly educational platform, the Smart Learning System cannot stop at being functional, but need focusing on efficiency and overall system performance. Thus, it must also adhere to a set of non-functional requirements that govern its performance, scalability, security, and overall user experience.

- **Scalability and Modularity:**

- The backend architecture must be designed as stateless, relying exclusively on external persistence layers for state management to ensure readiness for future distributed scaling.
- System design must be modular, with clear separation of components responsibility and dependency to facilitate future enhancements, maintenance, and potential integration of additional features or third-party services.

- **Resource Optimization and Latency Minimization:**

- Operating under the constraint of localized execution, the system must optimize memory and computational overhead, loading resource-heavy models into memory only when explicitly demanded by an active workflow.
- All resource must be efficiently released immediately after use to minimize the system's memory footprint and ensure responsiveness.
- System must be scheduled to be constantly running, minimizing downtime and waiting time for any task.
- All resource must be manageable, specifically for developers and system administrators, but crucial for maintaining system stability and performance.

- **Reliability and Determinism:**

- The system must mitigate the inherent unpredictability of generative AI by enforcing strict, programmatic data schemas for all internal system communications and decision-making logic.
- Complex analytical tasks, such as calculating knowledge gaps, must be executed via deterministic programmatic logic rather than relying on probabilistic generative approximations.

- **Observability and Traceability:**

- The system must maintain comprehensive telemetry across all automated workflows.
- Database state changes, including knowledge graph updates and student progress logs, must be monitored and saved to enable historical analysis and rollback capabilities.
- Every automated decision, context retrieval action, and resource consumption metric must be actively logged to provide a transparent audit trail for both system debugging and pedagogical evaluation.

- **User Experience and Interface Requirements:**

- User interface is focus on the simplicity and intuitiveness, ensuring that users of varying technical proficiency can navigate the system effectively.
- User interface must be responsive and accessible, adhering to modern web standards to ensure compatibility across a wide range of devices and screen sizes (phones, tablets, desktops).
- Client side shouldn't be pulled by heavy backend response latency. It should maintain a fluid, real-time interaction experience, minimizing latency in response generation and interface updates. Minimizing "user experience latency" regardless of system performance is a strategic design move to preserve conversational engagement as well as learning interest.

Chapter 4

Proposed Methodology and Paradigm

Building on the vision outlined in the requirements specification (Chapter 3), this research aims to propose a novel system that combines the strengths of classical Graph-based architectures with the generative capabilities of Large Language Models (LLMs). This chapter details the foundational methodology of our system.

4.1 Overall Pipeline

The proposed paradigm can be constructed and visualized as the below pipeline

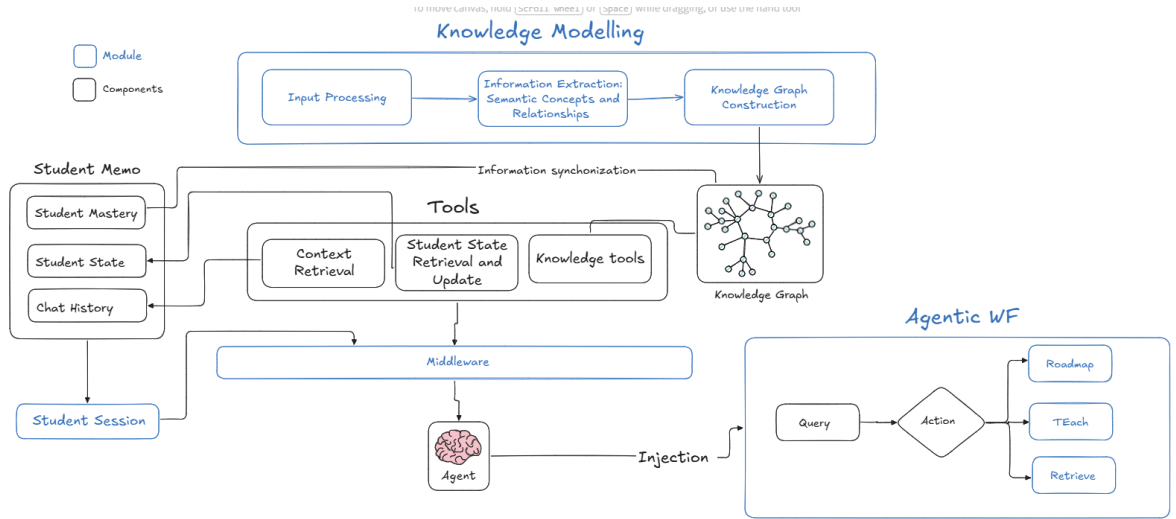


Figure 4.1: Macro workflow of the SmartEdu system, illustrating the Knowledge Modelling, Middleware, and Agentic Workflow orchestration.

A key security boundary is established at the system's entry point. Upon user authentication, the student's Session and Memo contexts are directly injected into the computational tools. As a result, the agents use these pre-configured tools instead of interacting directly with the database. This approach ensures strict data isolation and prevents the LLM from making unauthorized or destructive database transactions.

4.2 Knowledge Modelling: Educational Graph

This section proposes a hybrid methodology that anchors the generative flexibility of LLMs to the deterministic rigor of Graph Construction and Natural Language Processing. By formulating the educational domain as a Property Graph, the proposed system (SmartEdu) achieves a necessary synthesis of semantic richness and logical consistency, overcoming the unstructured chaos often associated with pure generative models.

4.2.1 Graph Definition

To construct a graph, we need to define and regulate it with a connected, rigorous structure. Simulating educational concepts, the graph contains ontologies as well as topic's relationship like prerequisite. The subsequent contents in this subsection explain the deterministic algebra representation and rules explanation of it.

4.2.1.1 Formal Property and Topological Definition

To accurately represent the vast, interconnected semantic structure inherent in educational curricula, our methodology employs a formal Property Graph formulation. Unlike simple directed graphs, a property graph allows both vertices and edges to possess complex internal states (properties). This multidimensionality is highly suitable for educational knowledge representation, where a concept is not just a node, but a container of definitions, mathematical formulas, and historical context.

We define the Educational Knowledge Graph (EKG) as a heterogeneous, directed property graph, formally denoted as:

$$G = (V, E, \Lambda, \Pi) \quad (4.1)$$

where:

- V is the set of vertices (nodes), representing distinct educational entities spanning multiple levels of abstraction.
- $E \subseteq V \times V$ is the set of directed edges, representing the semantic and structural relationships between entities.
- Λ is a finite, predefined set of labels, acting as ontological categories that strictly classify the nodes to prevent semantic drift.
- Π is a set of properties, expressed as key-value pairs $\pi_{key} = value$, associated with both nodes and edges to store metadata, raw textual content, and dynamically updated quantitative metrics.

The necessity of this mathematical definition lies in its ability to enforce data integrity. By treating the knowledge base as a set V and a mapping E , we can apply rigorous graph-theoretic algorithms (such as topological sorting and shortest-path routing) directly to the curriculum, effectively translating human pedagogical intuition into computationally solvable problems.

4.2.1.2 Ontological Hierarchy formulation

To accurately model the multi-scalar nature of educational curricula—ranging from entire academic disciplines down to specific explanatory sentences in a textbook—we artificially constrain the label set Λ such that $\Lambda = \{\text{CommunityNode}, \text{TopicNode}, \text{ConceptNode}, \text{RhetoricalNode}\}$. This rigorous ontological taxonomy ensures structural consistency across different domains:

- **CommunityNode:** Represents the highest level of educational aggregation. It functions as the macroscopic backbone of a course or a major disciplinary cluster, grouping broad fields of study (e.g., "Computer Science", "Artificial Intelligence").
- **TopicNode:** Represents a specific subject matter, module, or chapter nested strictly within a parent community (e.g., "Machine Learning", "Neural Networks").
- **ConceptNode:** The atomic unit of educational instruction. It represents distinct, indivisible ideas, theories, algorithms, or historical events (e.g., "Backpropagation", "Decision Tree").
- **RhetoricalNode:** Encapsulates the instructional function of the raw text. It contains specific properties $\pi_{role} \in \Pi$ that designate the rhetorical role of the information (e.g., *Definition*, *Application*, *Statement*, *Theorem*). This allows the system to distinguish between *what* a concept is and *how* it is explained.

By enforcing this strict hierarchy, the system prevents the "flattening" of knowledge, ensuring that the AI agent can distinguish between a broad topic that requires a multi-week syllabus and a specific concept that can be explained in a single paragraph.

4.2.1.3 Edge Semantics and DAG Constraints

The topological structure of G is governed by the relation types assigned to the edges $e \in E$, formulated as ordered pairs $e = (v_i, v_j, \lambda_e)$, where λ_e is the edge type. The system defines several core edge types to represent real-world pedagogical connections: *PART_OF*, *RELATED_TO*, and *PREREQUISITE_OF*.

The mathematical enforcement of the *PREREQUISITE_OF* edge is of paramount educational importance. In human learning, knowledge is inherently sequential; one cannot grasp Calculus without first mastering Algebra. To ensure logical learning progressions computationally, the sub-graph induced by all *PREREQUISITE_OF* edges must strictly form a **Directed Acyclic Graph (DAG)**.

Let $G_{prereq} = (V, E_{prereq})$ where $E_{prereq} = \{e \in E \mid \lambda_e = PREREQUISITE_OF\}$. The methodology mandates that no directed cycles exist within G_{prereq} :

$$\forall v \in V, \nexists \text{ path } P = (v, \dots, v) \text{ in } G_{prereq} \quad (4.2)$$

This acyclic constraint is not merely a theoretical preference; it is a foundational mathematical requirement. If a cycle were to exist (e.g., A is a prerequisite for B, B is a prerequisite for C, and C is a prerequisite for A), the student would be permanently trapped in a circular dependency, unable to begin learning any of the concepts. Enforcing the DAG constraint mathematically guarantees that algorithms like Topological Sorting can be successfully applied to generate optimal, conflict-free learning roadmaps.

4.2.2 Graph Construction

A major challenge in building an automated knowledge base is extracting structured relationships from unstructured educational text. Single-pass LLM extractions—where the model tries to identify concepts, classify them, and link them all at once—are extremely prone to failure. Multiple frameworks propose multiturn LLM synthesis, but costly and still not guarantee a solid, hallucination immunity graph as subsequence turns increase entropy and the context total amount. I propose a hybrid method of Dual LLM extraction combining with NLP based filtering and normalization, guarantee a consistent graph.

4.2.2.1 Dual Phase LLM Extraction

Dual Phase Extraction Pipeline is one that methodologically decouples top-down hierarchical decomposition from lateral relational mapping. Let $\mathcal{T}$ represent the continuous corpus of unstructured educational text, ingested from raw documents. The overall extraction objective is to discover an approximation function $\Phi: \mathcal{T} \rightarrow (V, E)$ that maximizes semantic fidelity. We mathematically decompose Φ into two sequential, deterministic mappings.

Phase 1: Top-Down Hierarchical Extraction (Skeleton Phase)

The primary objective of the first phase is to establish the taxonomic structure of the knowledge domain without the cognitive burden of cross-referencing. The LLM’s action space is strictly confined to identifying entities and their subsumption (parent-child) relationships.

We define the skeleton extraction function Φ_{skeleton} as:

$$\Phi_{\text{skeleton}} : \mathcal{T} \rightarrow (V_{\text{tree}}, E_{\text{tree}}) \quad (4.3)$$

where $E_{\text{tree}} \subset V_{\text{tree}} \times V_{\text{tree}}$ contains strictly hierarchical edges (e.g., *PART OF*).

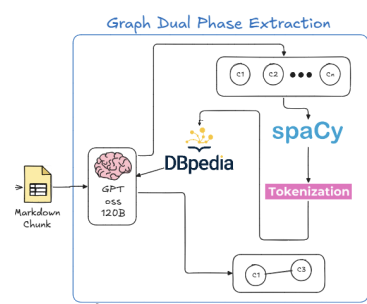


Figure 4.2: *Dual Phase Extraction*

To eliminate ontological drift, the LLM is forced to output data conforming to a rigid, predefined syntactic schema (e.g., strict JSON schema validation). The system parses this structural output to instantiate `CommunityNode` and `TopicNode` entities, ensuring that every atomic concept is firmly anchored to a macroscopic curriculum node.

Mathematically, this phase yields disjoint taxonomic trees. Because the LLM is only tasked with hierarchical assignment, the spatial complexity is bounded to $O(N)$ parent-child assignments. This drastic reduction in the problem space significantly mitigates the probability of generation errors. To complete the graph, phase 2 is executed subsequently.

Phase 2: Lateral Relational Mapping (Relation Phase)

With the hierarchical skeleton established and mathematically frozen in the database, the second phase focuses on inferring the non-hierarchical, semantic linkages across different branches of the tree. Given the isolated, immutable entity set V_{tree} , the relational extraction function Φ_{relation} is defined as:

$$\Phi_{\text{relation}} : (V_{\text{tree}}, \mathcal{T}) \rightarrow E_{\text{lateral}} \quad (4.4)$$

where E_{lateral} encompasses cross-cutting educational dependencies such as *PREREQUISITE_OF* or *RELATED_TO*.

By presenting the LLM with a predefined, validated list of concepts (V_{tree}) and tasking it *only* with edge generation, the problem space is reduced to a binary classification task over possible pairs. The final Educational Knowledge Graph is the formal union of these two sub-spaces:

$$G = (V_{\text{tree}}, E_{\text{tree}} \cup E_{\text{lateral}}) \quad (4.5)$$

To summarize, it is the operation of extracting potential candidate concepts as taxonomy trees and combine it to make a graph. This paradigm ensures high consistency in the generated graph. It bridges the gap between unstructured educational data and structured property graph by artificially constraining the LLM's action space, partially preventing hallucinations. By partially, there is a big problem: Phase 2 relies mainly on Phase 1 for a structured and stable output, but what if Phase 1 itself is not a good frame skeleton? To better reconstruct and verify the frame, we use deterministic algorithm and techniques

4.2.2.2 NLP-Based Concept Filtering and Verification

Although the Dual-Phase LLM Extraction (Section 4.2.2.1) significantly reduces hallucination by constraining the model's action space, the raw output of Phase 1 remains susceptible to surface-level inconsistencies: semantically equivalent concepts may appear under distinct lexical forms, and spurious or overly granular entities may be introduced without grounding in established knowledge. To address these issues, a deterministic NLP pipeline is interposed between Phase 1 and Phase 2. This pipeline operates exclusively on the candidate node set V_{tree} and produces a refined, verified set $V^* \subseteq V_{\text{tree}}$ that is subsequently passed to the Relation Phase.

The pipeline proceeds through four sequential steps.

Step 1 — Normalization and Tokenization. Each candidate concept label $v_i \in V_{\text{tree}}$ is first subjected to a standard text normalization procedure. This entails lowercasing, punctuation stripping, morphological lemmatization (reducing inflected word forms to their base lemma), and the removal of domain-agnostic stop words. The resulting form is then decomposed into a set of meaningful unigrams and bigrams, yielding a normalized token set $\mathcal{N}(v_i)$. Formally:

$$\mathcal{N}(v_i) = \text{tokenize}(\text{lemmatize}(\text{lower}(v_i))) \setminus \mathcal{S} \quad (4.6)$$

where $\mathcal{S}$ denotes the stop-word vocabulary. This normalization step establishes a canonical surface representation for every concept, ensuring that subsequent comparison operations are performed on semantically uniform token sets rather than raw, noisy strings.

Step 2 — External Validation via DBpedia. Following normalization, each candidate concept is submitted to the DBpedia Spotlight entity linking service. For a given concept v_i , the system issues a query against the DBpedia knowledge base and retrieves the top-ranked entity annotation along with its associated confidence score $\sigma(v_i) \in [0, 1]$. A concept is retained in the verified set only if:

$$\sigma(v_i) \geq \tau_{\text{dbp}} \quad (4.7)$$

where τ_{dbp} is a predefined acceptance threshold. Concepts failing this criterion are classified as either hallucinated constructs or excessively fine-grained pseudo-entities that lack grounding in established encyclopedic knowledge, and are subsequently removed from V_{tree} . This external validation step introduces a layer of epistemic accountability, anchoring the knowledge graph to verifiable, real-world concepts rather than model-generated approximations.

Step 3 — Implicit Clustering via Token Overlap. Upon completion of filtering, the retained concepts undergo pairwise token similarity analysis. The Jaccard similarity between two concept representations is computed as:

$$J(v_i, v_j) = \frac{|\mathcal{N}(v_i) \cap \mathcal{N}(v_j)|}{|\mathcal{N}(v_i) \cup \mathcal{N}(v_j)|} \quad (4.8)$$

This measurement serves two complementary functions. First, concept pairs satisfying $J(v_i, v_j) \geq \tau_{\text{merge}}$ are treated as near-duplicates and consolidated into a single canonical node, thereby eliminating redundancy introduced by paraphrastic variation in the source text. Second, and more significantly, concepts that share a non-trivial common token subset — without necessarily crossing the merge threshold — are grouped into an *implicit cluster*. The intersection $\mathcal{N}(v_i) \cap \mathcal{N}(v_j)$ is used to infer a shared parent topic label; for instance, the concepts “Machine Learning Algorithms” and “Machine Learning Optimization” both yield the shared token “Machine Learning”, from which a `TopicNode` bearing that label is derived. These implicitly discovered topic nodes are then cross-validated against the hierarchical skeleton produced by Phase 1 to either confirm existing `TopicNode` entries or surface structural gaps in the original extraction.

Step 4 — Verified Node Caching. Once a concept node has successfully traversed Steps 1 through 3 — normalized, externally validated, and cluster-assigned — its canonical representation is persisted in a key-value cache, indexed by its normalized token set $\mathcal{N}(v_i)$. This cache serves three purposes. First, it ensures *idempotency*: if the same concept surface form is encountered across multiple ingested documents, it resolves deterministically to the same canonical node without re-executing the NLP pipeline. Second, it supports *incremental graph construction*: as new educational materials are added to the corpus over time, only genuinely novel concepts need to traverse the full verification pipeline, while previously verified nodes are retrieved directly from cache. Third, it provides the Relation Phase (Phase 2) with a stable, pre-validated entity vocabulary, eliminating the need for redundant lookups during edge generation. The final verified and deduplicated set V^* is then passed forward as the immutable input to Φ_{relation} .

4.3 Teaching Assistance

The knowledge graph G serves as the static, structural foundation of the system. However, real-time interaction with a human learner demands a dynamic, adaptive reasoning engine capable of navigating G to produce personalized educational responses. A single monolithic language model, no matter how capable, cannot reliably handle the diverse reasoning tasks involved in tutoring—from classifying student intent, to retrieving factual content with minimal cost and latency, to generating structured lectures, to evaluating learning outcomes. Each of these tasks requires a different workflow design.

To address this challenge, the proposed system decomposes the tutoring process into a multi-agent workflow, where each reasoning step is handled by a dedicated agent node with its own tailored prompt, tools, and output schema. The following subsections describe the core architectural decisions that make this decomposition both practical and effective: its applied harnessing as well as workflow design.

4.3.1 Agent Harnessing

Rather than relying on a single, prompt-heavy LLM to manage all tutoring logic, the Teaching Assistant is structured as a collection of specialized agent nodes coordinated through what we term an *Agent Harness*. This harness governs how each node receives its instructions, what context it can access, and what format its output must follow. The design philosophy, called by myself the ‘‘Single Paradigm’’, is straightforward — each agent node should have exactly one responsibility, exactly one instruction set with one consistent context, and exactly one input/output structure/schema definition.

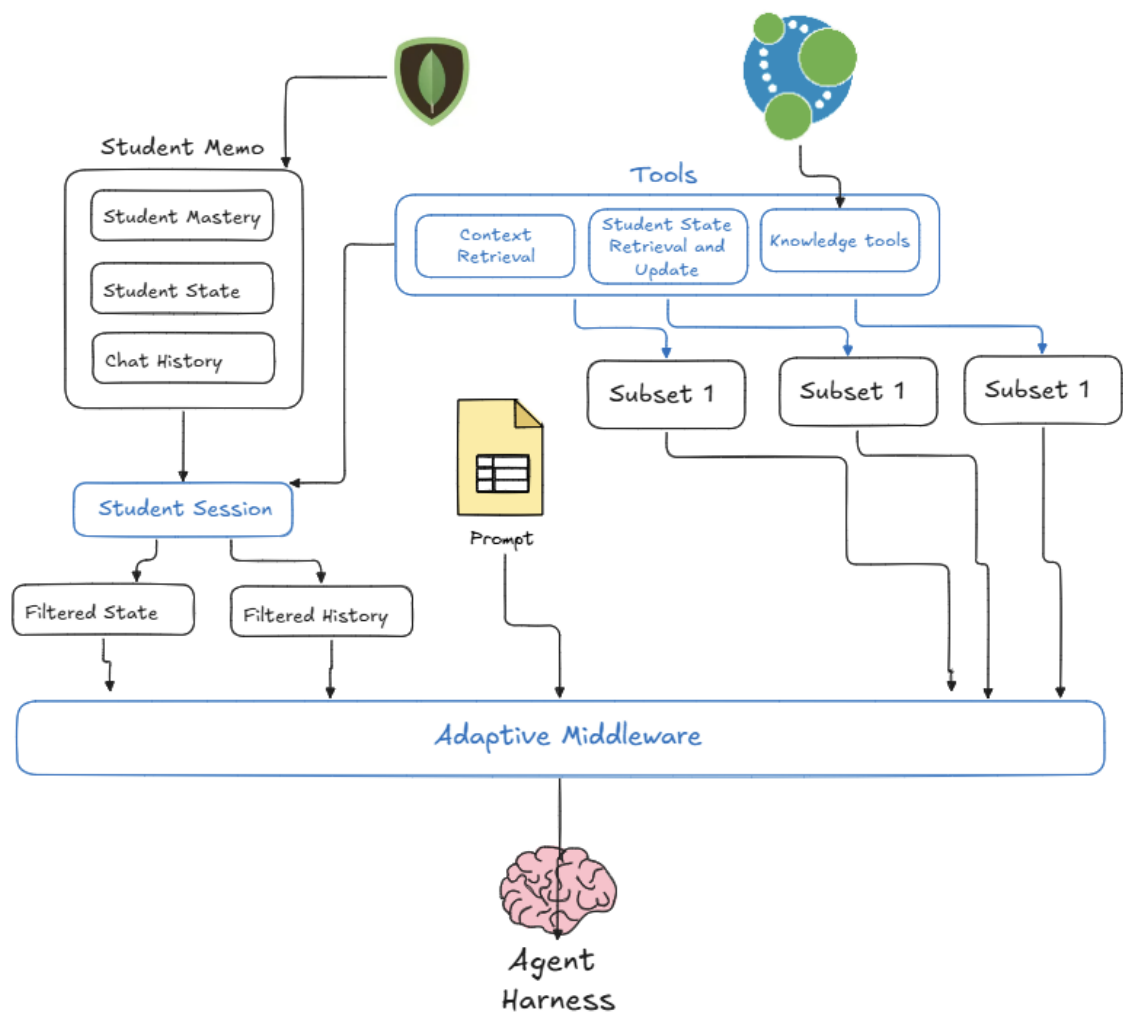


Figure 4.3: *Agent Harness architecture demonstrating the Adaptive Harnessing based on Middleware.*

The following paragraphs detail the key techniques that enable this architecture, as visualized fully in Figure 4.3

Chat Summarization and Memory Management

A persistent challenge in multi-turn conversational AI is the accumulation of context. As a tutoring session progresses, the raw chat history grows rapidly, filled with verbose tool outputs, intermediate reasoning traces, and previous lecture content. Feeding this entire history into every agent call would quickly exhaust the model's context window and introduce significant noise that degrades reasoning quality.

To mitigate this problem, the system employs a sliding-window memorization mechanism. Instead of preserving the complete raw transcript, each conversational turn is condensed into a structured triplet of the form: *query* → *thought* → *answer summary*. Only a fixed number of these summarized entries (default at 8) are retained in the active session memory at any given time. Older entries are dropped from the window as new ones arrive, ensuring that the memory footprint remains bounded regardless of session length.

This summarized history is then presented to different agents in different formats depending on their needs. Lightweight classification nodes—such as the intent router—receive only the heading summaries of previous interactions (a mode referred to as *skim*). In contrast, nodes that require deeper contextual awareness—such as lecture generation or evaluation—receive the full summarized messages. This selective exposure ensures that each agent receives precisely the amount of historical context it needs: enough to maintain conversational coherence, but not so much that it overwhelms the model's reasoning capacity.

4.3.1.1 Tool Abstraction Design

The SmartEdu system operates on top of a complex data infrastructure comprising a graph database (Neo4j), a vector store (Milvus), a document store (MongoDB), and an object storage service (MinIO). Allowing an LLM agent to interact directly with these systems would introduce severe risks: the model could construct malformed queries, access unauthorized data, or perform destructive write operations.

To prevent these issues, the system introduces a tool abstraction layer that sits between the agents and the underlying databases. Each tool exposes a single, well-defined function—for example, resolving a concept name to its graph identifier, retrieving all rhetorical content associated with a node, or extracting a specific page from a stored PDF document. The agent interacts exclusively with these high-level tool interfaces and has no direct access to raw database connections or query languages.

Critically, these tools are pre-configured with the active student's session context at the beginning of each request. The student's identity, learning history, and session scope are injected into the tools before the agent ever sees them. From the agent's perspective, calling a tool named `entity_finder` with a concept name is a simple, self-contained operation. The agent does not need to know which database engine is being queried, how the algorithm works, or how the results are filtered by student scope. This separation achieves two goals simultaneously: it simplifies the agent's decision space (reducing the probability of incorrect tool calls), and it enforces strict data isolation (preventing cross-session leakage or unauthorized mutations).

4.3.1.2 State—based Harnessing

In a conventional single-agent architecture, the entire system prompt, all available context and tools are ninded once at the beginning of the conversation and remain static throughout. This approach is quite common due to simplicity and the popularity of Large language Model with huge context, but becomes increasingly problematic as the complexity of the task exceed the context window, because the model must simultaneously bind and remember instructions for routing, retrieval, evaluation, and response generation.

The Dynamic Harness addresses this limitation through state-dependent context injection. Each node in the workflow graph is associated with its own prompt template, and these templates are populated dynamically at runtime based on the current workflow state. When the system reaches a retrieval node, the agent receives only retrieval-specific instructions along with the relevant query and entity data. When it reaches an evaluation node, the agent receives evaluation criteria and the recent interaction history. The student's current learning position, mastery profile, and active resource are injected only into nodes where they are pedagogically relevant.

Furthermore, synthesis nodes—those responsible for producing the final student-facing response—receive an additional injection of prior reasoning from upstream nodes. For instance, the final advisory node in the Roadmap workflow receives not only the proposed learning path but also the critique generated by the evaluator node that preceded it. This chained injection ensures that the final output reflects the accumulated reasoning of the entire pipeline, rather than being generated in isolation. The result is a system where each agent operates with a minimal, purpose-built context window, yet the overall pipeline produces coherent, multi-step reasoning.

4.3.1.3 LangChain Middleware Architecture

The dynamic per-node behavior described above requires a mechanism that can intercept and modify agent execution at each step of the workflow. Standard agent frameworks—including earlier versions of LangChain, as well as alternatives such as CrewAI and AutoGen—typically operate with a fixed configuration: the agent's tools, system prompt, and output format are defined once at initialization and remain constant throughout execution. This rigidity makes it difficult to change the agent's behavior dynamically based on which node is currently active.

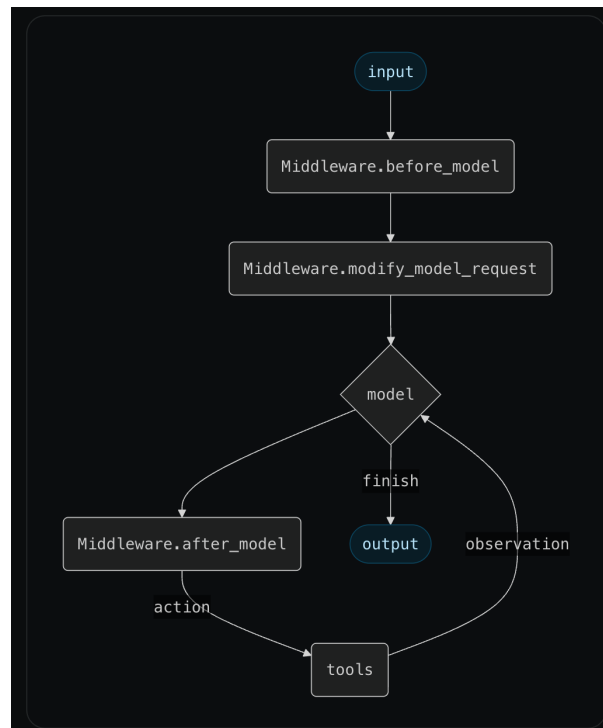


Figure 4.4: *Langchain Middleware. Referenced from [Langchain Documents](#)*

The Middleware abstraction, introduced to LangChain in later middle of 2025, provides a solution to this problem. A middleware component is a composable interceptor that hooks into the agent’s execution loop at the model-call level. Specifically, it exposes an asynchronous method (`awrap_model_call`) that receives the model request before it is sent to the LLM and can modify, augment, or redirect that request before passing it to the actual handler. This design pattern is analogous to middleware in web server frameworks, where request-processing logic is composed as a chain of interceptors rather than a monolithic handler.

The key advantage of this architecture over role-based multi-agent frameworks (such as CrewAI’s crew-of-agents or AutoGen’s conversable agents) is granularity. In role-based systems, each agent is a fully independent entity with its own model instance, prompt, and tool set. Changing the agent’s behavior requires spawning an entirely new agent. With middleware, a single underlying agent instance can exhibit different behaviors at different workflow nodes, because the middleware dynamically modifies its configuration on each invocation. This eliminates the overhead of maintaining multiple redundant model connections while preserving the flexibility of per-node specialization.

4.3.1.4 Agentic Cooperation into Workflow

Building on the LangChain middleware foundation, the proposed system introduces a custom middleware component termed the *Adaptive Middleware*. This middleware maintains an internal registry that store all of workflow node names. When the agent is invoked at a particular node, the middleware inspects the current node identifier in the workflow state, finding its registered harnessing protocol of the node, to reconstruct the agent before truly invoke the agent for the specific tasks that node.

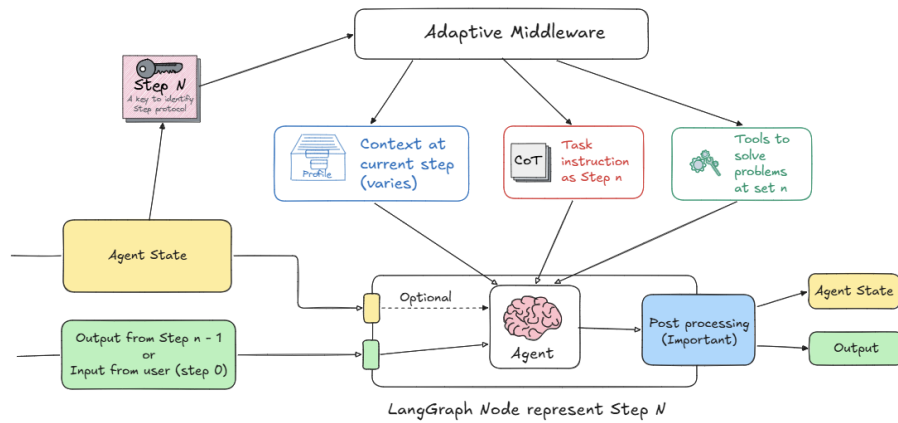


Figure 4.5: How agents cooperate into each node of the workflow

This design offer 4 novel benefits:

1. Single Paradigm is applied when workflow node is registered with its own harnessing protocol, preventing the confusion that arises when a model must choose among multiple possible reasoning, requirements and methods.
2. Because the middleware handles schema routing automatically, the prompt templates for each node can focus exclusively on the reasoning task at hand, without needing to include format-selection instructions.
3. By constraining the output space to a predefined set, the system significantly reduces the probability of the model generating structurally invalid or semantically drifted responses. Each agent node operates within a tightly bounded action space—one task, one context, one schema—which collectively yields a more reliable and debuggable multi-step reasoning pipeline.
4. With the registry techniques, agents are not hardcoded in nodes during initial. This allow easy behaviour debug from the central registry, making agentic design more flexible and scalable.

4.3.2 Agentic Workflow

The Teaching Assistant's reasoning process is organized as a hierarchical state machine. At the top level, a router node classifies each incoming student query into one of five intent categories and dispatches it to the corresponding sub-workflow. Each sub-workflow is itself a directed graph of specialized nodes, where each node performs a single reasoning or data-retrieval step. Upon completion, every sub-workflow passes its results to a synthesis node that formats the final student-facing response and persists the session state. Figure 5.3 illustrates this top-level architecture.

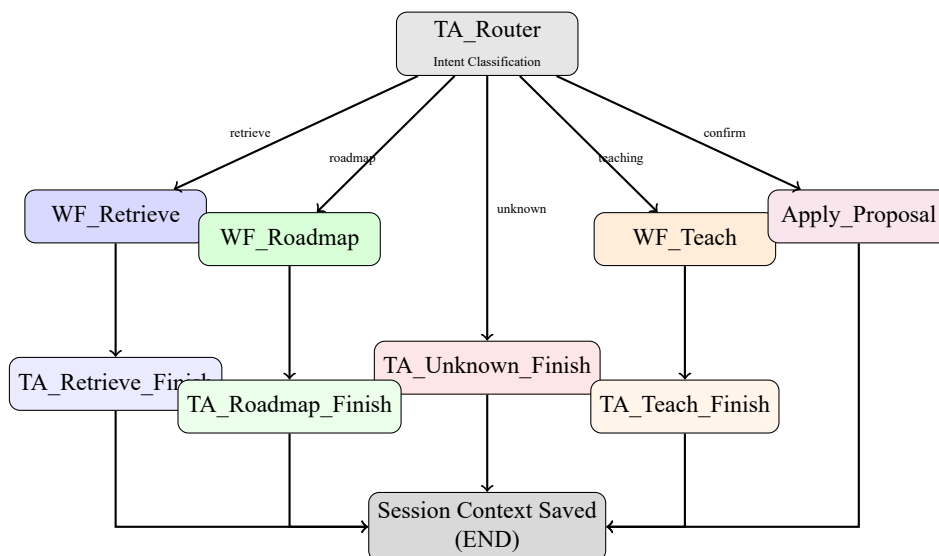


Figure 4.6: Top-level SmartEdu state machine detailing the hierarchical intent routing and background persistence synchronization.

Intent Router

The entry point of every student interaction is the Intent Router. This node receives three inputs: (1) the student's current learning state (active position, pending proposals), (2) a summarized view of recent chat history (*skim* mode), and (3) the raw query. It classifies the query into exactly one of five intents: *retrieve*, *roadmap*, *teaching*, *confirm*, or *unknown*. The classification uses constrained generation (temperature zero, bounded output length) to produce a single word, without any tool invocation. This ensures minimal latency at the routing step.

4.3.2.1 Workflow: Roadmap

The Roadmap workflow addresses the question: *what should the student learn next, and in what order?* Unlike factual retrieval, this requires pedagogical reasoning—comparing the student's current mastery against the structural dependencies in the knowledge graph to produce a feasible study plan.

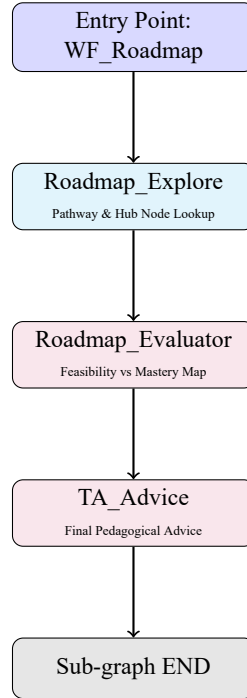


Figure 4.7: Roadmap sub-graph. The proposed path is queried from the EKG, critiqued against the student's mastery profile, and generated with bespoke pedagogical advice.

Node: Roadmap_Explore. A well-constructed knowledge graph is inherently dense and interconnected: a single course may contain hundreds of concept nodes linked by prerequisite, hierarchical, and associative edges. This density, while beneficial for semantic coverage, presents a navigation problem. Without a method to distinguish foundational concepts from peripheral ones, any attempt to generate a learning path would either overwhelm the student with every available node or arbitrarily select concepts without structural justification.

To address this, the exploration node applies *Out-Degree Centrality* as a structural heuristic to locate *backbone nodes*—the most foundationally connected concepts in a course. For a given node $v \in V$, the semantic out-degree $C_{deg}(v)$ is defined as:

$$C_{deg}(v) = \sum_{\substack{w \in V \\ (v,w) \in E_{sem}}} 1 \quad (4.9)$$

where $E_{sem} = \{e \in E \mid \lambda_e \neq \text{CONTENT}\}$ excludes purely content-bearing edges. Nodes satisfying $C_{deg}(v) \geq k_{min}$ are classified as *hub nodes*—concepts that serve as prerequisite or structural anchors for many downstream topics. These hub nodes collectively form the course backbone $\mathcal{B} = \{v \in V \mid C_{deg}(v) \geq k_{min}\}$.

However, simply ranking all hub nodes by centrality and presenting them as a flat list would discard the sequential ordering that makes a learning path pedagogically valid. A student must encounter foundational hubs before advanced ones, and the path must not skip over intermediate dependencies. To preserve this order while still selecting only the most important nodes, the system applies a *sliding window sampling* strategy over the topologically sorted backbone.

Let $\mathcal{B}^* = (b_1, b_2, \dots, b_n)$ be the ordered sequence of backbone nodes, sorted by their topological position in the prerequisite sub-graph G_{prereq} . A sliding window of fixed size w advances over $\mathcal{B}^*$ with step size s . Within each window $W_i = (b_j, b_{j+1}, \dots, b_{j+w-1})$, the system selects the top- k nodes ranked by C_{deg} :

$$\text{Selected}(W_i) = \text{top-}k(\{b \in W_i\}, C_{deg}), \quad k = 2 \quad (4.10)$$

The final summarized path is the ordered concatenation of selected nodes across all windows:

$$P_{\text{roadmap}} = \text{Selected}(W_1) \oplus \text{Selected}(W_2) \oplus \dots \oplus \text{Selected}(W_m) \quad (4.11)$$

This mechanism offers two key guarantees. First, the topological sort on G_{prereq} ensures that prerequisite concepts always appear before dependent ones, preserving the logical learning order. Second, the windowed selection prevents the path from clustering around a single dense region of the graph or jumping erratically between distant topics—each window captures the locally most important concepts within its segment of the curriculum, producing a path that is both comprehensive and sequential.

The node is equipped with three graph exploration tools: `recommend_new` (ranking nodes by semantic out-degree), `course_backbone` (extracting hub nodes and inter-hub relationships within a course), and `course_relevance` (detecting cross-course prerequisite dependencies). The output is a structured object containing the recommended starting node, the learning goal, and the ordered list of hub nodes forming the proposed path.

Node: Roadmap_Evaluator. The proposed path from the exploration node is not presented directly to the student. Instead, it passes through an evaluation step that examines feasibility against the student's current mastery profile. The evaluator performs three checks: a *necessity audit* (is each hub node truly essential?), a *prerequisite check* (does the student have recorded mastery for all required foundations?), and a *cognitive load assessment* (does the sequence avoid placing too many difficult concepts in immediate succession?). This node operates without tools—it is a pure reasoning step that produces a feasibility judgment and a critique narrative.

Node: TA_Advice. The advisory node reconciles the proposed path with the evaluator's critique. If prerequisite gaps were found, it recommends addressing those foundations first. If the path is feasible, it specifies the exact starting concept and provides concrete guidance. The output contains the finalized learning steps and pedagogical advice. This three-stage pipeline—explore, evaluate, advise—mirrors how a human tutor would plan: survey the landscape, assess readiness, then recommend.

Synthesis: TA_Roadmap_Finish. The synthesis node packages the advisory output into a formal *Learning Proposal*. Unlike Retrieve proposals (which are applied automatically), Roadmap proposals require explicit student confirmation before updating the session state. This design reflects the higher stakes of path-level decisions: changing a student's entire learning trajectory warrants informed consent. The proposal is stored as a pending item; when the student confirms, the router classifies this as a *confirm* intent, and the system commits the path.

4.3.2.2 Workflow: Retrieve

The Retrieve workflow answers factual questions by searching the Educational Knowledge Graph. Its design uses a two-tier retrieval strategy: a fast initial lookup that satisfies most queries, followed by an optional deep analysis that activates only when the system detects a significant knowledge gap.

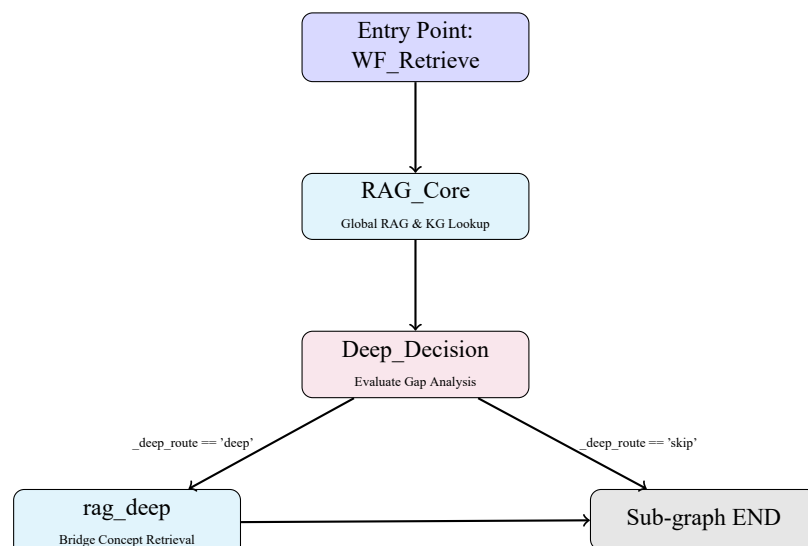


Figure 4.8: Retrieval sub-graph. The flow routes dynamically to deep verification of prerequisite concepts if potential knowledge gaps are identified.

Node: RAG_Core. The first node performs the primary knowledge graph lookup. The agent follows a strict Chain-of-Thought procedure: (1) resolve the concept name to a graph entity identifier using `entity_finder` (which queries Wikidata for a canonical ID, then falls back to direct name matching), (2) retrieve all educational content associated with that entity using `rhetorical_retriever` (organized by rhetorical role: definition, formula, theorem, example), and (3) submit findings through the designated output schema. The agent is limited to four tool calls to prevent runaway exploration.

Node: Deep_Decision. After the initial retrieval, this node evaluates whether the query requires deeper analysis. It receives the RAG_Core results, the original query, and the student's current position. Operating without tools, it classifies the situation as *DEEP* (the query involves mechanisms or concepts distant from the student's position) or *SKIP* (the basic retrieval suffices). This classification controls the conditional routing to the next node.

Node: rag_deep. When activated, this node performs a structural neighborhood analysis of the retrieved entity. Using the `edge_explorer` tool, it traverses multi-hop relationships (excluding content edges) to discover parent concepts, prerequisite dependencies, and semantically related nodes. The central objective is to identify *bridge concepts*—prerequisite knowledge the student needs before fully understanding the queried topic.

The detection of knowledge gaps relies on the same centrality-based framework introduced in the Roadmap workflow (Section 4.3.2.2). Specifically, the agent compares the student's current position against the topological neighborhood of the target concept. Bridge concepts are those nodes that lie on the prerequisite paths between the student's position and the target, ranked by their C_{deg} values. The output includes a knowledge gap score $\gamma \in [0, 1]$ (0 = no gap, 1 = significant gap) and a list of bridge concept nodes. If bridge concepts are identified, the node constructs a *bridge* proposal suggesting that the student's learning path be adjusted to cover the prerequisites first.

Synthesis: TA_Retrieve_Finish. The final node synthesizes all accumulated results—core retrieval content, deep analysis findings, and any bridge proposals—into a student-facing response. If the retrieval found no relevant content, the agent acknowledges this honestly rather than fabricating an answer. The output follows the standard response schema containing a summary, the full formatted message, and an optional front-end navigation action.

4.3.2.3 Workflow: Teach

The Teach workflow is the most structurally complex sub-workflow and the only one that directly manages the active learning loop: delivering lectures, assessing understanding, and determining when to advance.

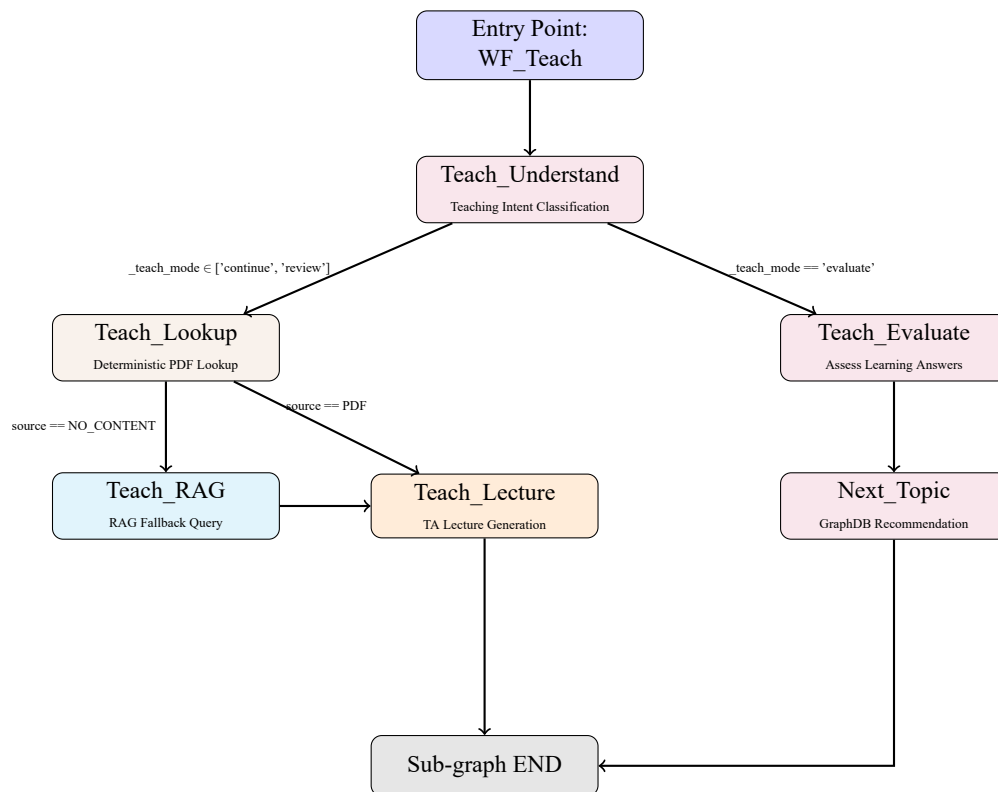


Figure 4.9: Teaching sub-graph flow detailing deterministic content lookup, RAG fallback pathways, active instruction delivery, and adaptive topic progression.

Node: Teach_Understand. This entry node classifies the student’s query within the teaching context into one of three sub-intents: *review*, *continue*, or *evaluate*. When ambiguous, the default is *continue*, ensuring the teaching process moves forward. The classification determines routing: *review* and *continue* proceed to content lookup, while *evaluate* routes directly to the assessment node.

Node: Teach_Lookup. To gather the teaching material for the current concept, this node employs an iterative *Reason-Act* (ReAct) loop. Rather than making a single retrieval call that may return incomplete or irrelevant content, the agent repeatedly reasons about what information is still needed and calls the appropriate tools until it judges the collected context to be sufficient. Figure ?? illustrates this iterative process.

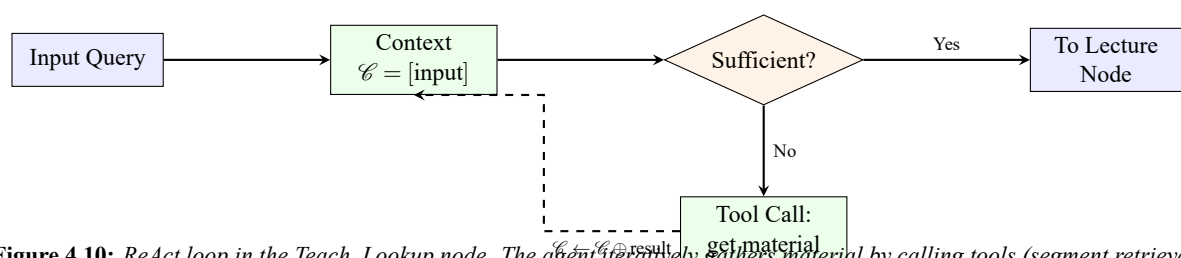


Figure 4.10: ReAct loop in the Teach_Lookup node. The agent iteratively gathers material by calling tools (segment retrieval, page extraction) and appending results to the running context $\mathcal{C}$. The dashed arrow represents the context concatenation feedback. The loop terminates when the agent judges the collected content to be sufficient for lecture generation.

The tools available at this node include `get_concept` (resolving the current concept to its *hard reference*—a structured pointer containing the document identifier and page number in the original course material) and `get_pages` (extracting specified pages from the stored PDF). The agent reads the retrieved material, evaluates whether it covers the concept adequately, and either proceeds to lecture generation or issues additional tool calls to fetch neighboring pages or related segments. If no hard reference exists, the flow is routed to a RAG fallback node that reuses the retrieval pipeline from the Retrieve workflow (Section 4.3.2.1).

Node: Teach_Lecture. The lecture generation node receives the collected source content, the student’s current concept position, previously completed concepts, and the recent chat history. In *continue* mode, the agent teaches the concept in depth, breaking the lecture into sections and posing challenge questions. In *review* mode, it focuses on recall and retention testing. The instruction adapts to the student’s Bloom’s taxonomy level: at lower levels

(Remember, Understand), the focus is on definitions and basic recall; at higher levels (Apply, Analyze, Evaluate, Create), the focus shifts to application, reasoning, and synthesis.

Node: Teach_Evaluate. When the student submits an answer, this node analyzes the recent interaction history to judge mastery. The agent identifies the questions posed during the session, assesses whether the student's responses reflect genuine understanding or surface-level recall, and renders a pass or fail judgment. This node operates without tools, relying solely on the conversational evidence.

Node: Next_Topic. If the evaluation passes, this node selects the next learning target. It retrieves neighboring concept nodes from the graph, ranked by out-degree centrality C_{deg} , and presents the candidates to the agent. The agent selects one to three appropriate next topics based on prerequisite relationships and the student's performance. The selected nodes are packaged as an *advance* proposal, which—unlike Roadmap proposals—is applied automatically. If the evaluation fails, the node returns a “STAY” action and the student's position remains unchanged.

Synthesis: TA_Teach_Finish. The synthesis node applies any auto-approved proposals (updating the student's graph position), formats the lecture or evaluation feedback into a student-facing response, and—if the source content came from a PDF—attaches a navigation action directing the front-end to display the corresponding document page alongside the generated lecture.

Chapter 5

System design and implementation

This chapter presents the concrete design and implementation of the SmartEdu system, translating the methodology outlined in Chapter 4 into a functioning software implementation. Specifically, it covers the following aspects:

- **System Architecture and Design Patterns:** A high-level description of the system’s architecture, its components, their responsibilities, and how they interact. Moreover, it explains the design patterns and software engineering principles that lie behind and support the architecture design, ensuring the system’s modularity, scalability, and maintainability.
- **Core Module:** The foundational services and utilities that support all other modules, including LLM management, database clients, configuration handling, and shared data schemas.
- **Student Module:** The implementation of authentication, session management, and the Student Tracker facade that abstracts away persistence details while enforcing privacy constraints.
- **Knowledge Module:** The offline pipeline for constructing the educational knowledge graph from raw course materials, including extraction, transformation, and loading stages.
- **Teaching Assistant (TA) Module:** The real-time agentic workflow that processes student queries, interacts with the knowledge graph, and generates educationally grounded responses.

5.1 System Architecture

5.1.1 Overall Architecture and Module Responsibilities

The SmartEdu backend is a monolithic FastAPI application composed of five functionally distinct, loosely coupled modules. This *Modular Monolith* design was deliberately chosen over a microservices architecture at this stage: consolidating modules into a single process eliminates network-induced latency between the components of the multi-step agentic workflow, while the strict dependency inversion between modules (discussed in Section 5.1.3) ensures that each module can be extracted into an independent microservice with no changes to its internal business logic.

Table 5.1: System Module Summary

Module	Primary Responsibility	Key Classes
core	Shared infrastructure: LLM engine, all DB repository clients, configuration, shared schemas.	CoreLLMEngine, GraphDB, MilvusDB, MinioDB
student	Authentication, session lifecycle, unified learning-state facade.	Student_Tracker, StudentSession, Memo
knowledge	Offline EKG construction pipeline from raw course materials.	KnowledgeModule, CourseIngestionService
TA	Real-time agentic workflow; generates pedagogically grounded responses.	TAModule, SmartEdu, AgentInjector
tracing	Per-session observability; async trace buffering and Langfuse flush.	AgentTracer, TraceWriter

The inter-module dependency graph is strictly acyclic: `core` is the sole shared foundation depended on by all other modules; `student`, `knowledge`, and `TA` are peer modules that do not import one another; `tracing` is consumed exclusively by `TA`. This topology is illustrated in Figure ??.

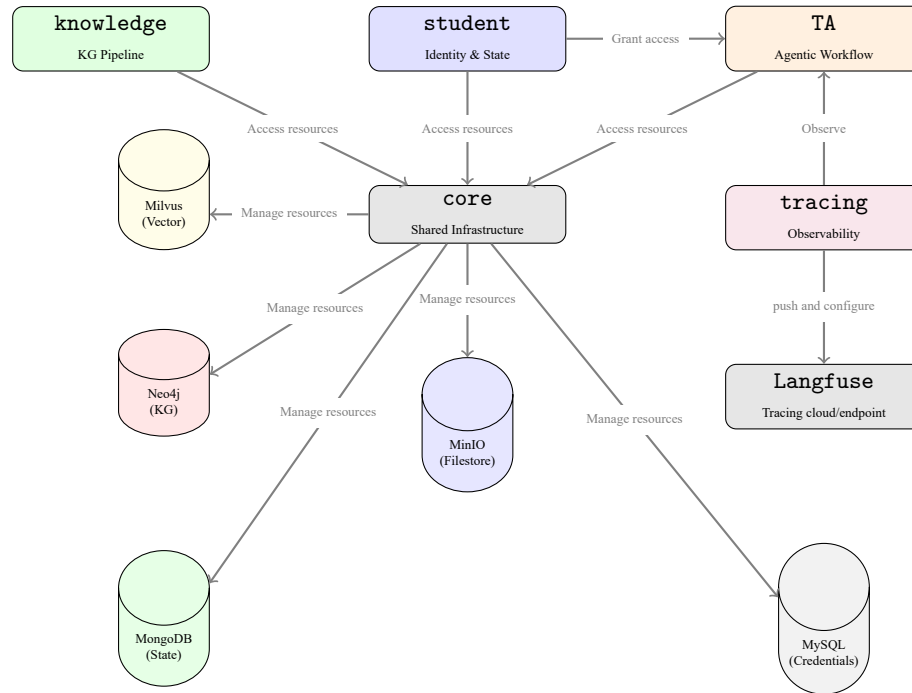


Figure 5.1: Modular Architecture Overview of SmartEdu

5.1.2 Singleton via Lifespan Management

A central engineering challenge is ensuring that expensive, long-lived resources — LLM model instances and database connection pools — are initialized exactly once and shared safely across all concurrent HTTP requests. Constructing these resources per-request would incur prohibitive latency and memory exhaustion.

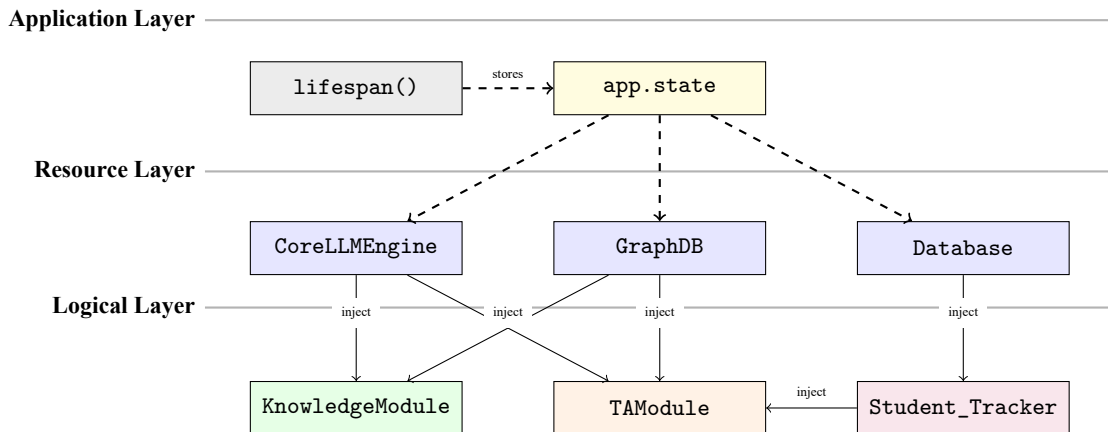


Figure 5.2: Ownership graph in lifespan. Shared resources are injected into domain modules

The system resolves this through FastAPI's `lifespan` context manager (`core/api/life_span.py`), which implements the *Singleton* pattern at the application scope. During startup, `lifespan` constructs every shared resource once, stores it on `app.state`, and yields control to the server. On shutdown, it performs explicit teardown. The class diagram in Figure ?? illustrates the construction and ownership relationships.

This approach yields three concrete benefits. First, unified resource control: all configuration (database URIs, model names, API keys) is read from a single `.env` file at startup, guaranteeing that the entire application operates on a single coherent configuration snapshot for its entire lifetime. Second, RAM efficiency: heavy resources (LLM weights, connection pools) are allocated once and never duplicated — multiple modules receive *references* to the same object, not copies. Third, clean shutdown: the `lifespan`'s teardown phase ensures all connections are explicitly closed, preventing resource leaks.

5.1.3 Dependency Injection via `core/dependencies.py`

Storing singletons on `app.state` is necessary but insufficient for clean architecture. Allowing route handlers to access `app.state` directly would tightly couple business logic to the ASGI framework internals, making handlers untestable in isolation.

The system resolves this through a dedicated *Dependency Injection* (DI) layer in `core/dependencies.py`. This file defines thin *provider functions* — one per injectable resource — that each return the corresponding singleton from `app.state`. Route handlers declare their dependencies as typed parameters annotated with `Depends()`, and FastAPI's IoC container resolves and injects the correct instance at request time.

This indirection yields three engineering benefits:

- **Decoupling of business logic:** Route handlers are pure functions of their typed inputs, completely unaware of how their dependencies are constructed.
- **Swappable implementations:** Replacing the LLM provider (e.g., local Ollama to a cloud API) requires modifying only the provider function and the lifespan initializer — not a single route handler.
- **Testability:** The test suite overrides any provider via `app.dependency_overrides`, enabling full unit testing without live database or LLM connections.

5.1.4 MVC-Aligned Module Structure and Scalability to Microservices

As proposed in Section ??, the system is designed to support complex stateful workflows while remaining strictly modular. Each domain module follows an MVC-aligned internal layout that enforces a clean separation of concerns:

- **Model (Schema):** Pydantic models in `core/schema/wf_state.py` act as the system-wide data contract. No module constructs ad-hoc dictionaries to communicate state; all data is typed and validated through shared classes such as `ConceptNode`, `StudentState`, `AgentState`, and `TAOutput`.
- **Controller (API Router):** Each module exposes its functionality through a dedicated FastAPI router (e.g., `TA/api/route.py`). The router handles only HTTP-level concerns: parsing requests, invoking the service layer, and serializing responses. It contains no business logic.
- **Service (Domain Module):** Domain classes (`TAModule`, `KnowledgeModule`) encapsulate all business logic. They receive infrastructure dependencies exclusively via constructor injection and communicate with the controller only through shared Pydantic schemas.

This strict layering has a crucial architectural consequence: because no module's business logic contains direct imports of another module's internal classes — only shared schemas from `core` — each module can be extracted into an independent microservice by simply changing its entry point. The DI mechanism is replaced by an HTTP client call, and the Pydantic schemas serve as the API contract, with zero modification to domain logic. This makes the system *scalable to microservices* by architectural design.

5.2 Core Module

The `core` package is the dependency foundation of the entire system. It owns no domain business logic; instead, it supplies the primitive, reusable services and shared data contracts upon which all domain modules are built. As established in Section 5.1.2, all `core` resources are constructed once during the lifespan phase and injected by reference.

5.2.1 LLM Engine: Multiton and Profile-Based Configuration

`CoreLLMEngine` (`core/llm/llm_engine.py`) is the central gateway for all LLM interactions. It implements the *Multiton* pattern — a generalization of Singleton where a fixed, named pool of instances is managed. Internally it maintains a `Dict[str, ChatOllama]` keyed by a profile name (e.g., "TA", "RAG", "Generator"). When an agent requests a model via `_get_llm("TA")`, the engine checks its internal instance cache. If the instance does not exist, it constructs and caches a new `ChatOllama` object from the corresponding `LLMProfile` defined in `core/llm/config.py`; subsequent calls return the cached instance.

Each `LLMProfile` dataclass encapsulates all operational parameters for one agent: `model_name`, `temperature`, context window size (`num_ctx`), and a `max_retries` budget. For example, the "TA" profile uses a small 4B model at higher temperature for conversational fluency, while the "graph" profile uses a large 397B cloud model at low temperature for deterministic structured extraction. This *lazy instantiation* ensures that a model

is only loaded into GPU memory when it is first needed, optimizing resource utilization when some agents are inactive.

The `invoke_with_retry` method abstracts the full LangChain invocation pipeline: it composes a `prompt_template`, the resolved `ChatOllama` instance, and a Pydantic output parser into a single executable chain, then runs it with automatic retries on `OutputParserException`. This centralization ensures that all retry logic, error handling, and chain composition follow one consistent implementation across all agents and all modules.

5.2.2 Configuration Management: Dataclass-Based Centralization

All system configuration is expressed as Python `@dataclass` objects in `core/config.py`. Each subsystem has a dedicated configuration class — `Neo` for the knowledge graph database, `Mil_conf` for the vector store, `Emb_conf` for the embedding model, `TA_conf` for the agent definitions. Environment variables are read exactly once at module import time via `python-dotenv`, which loads values from a centralized `.env` file. All dataclass fields default to sensible values if the corresponding variable is absent.

This design provides two key advantages over string-based configuration dictionaries. First, configuration objects benefit from static type checking and IDE autocompletion, reducing the risk of silent key-name typos. Second, the single `.env` file acts as the sole source of truth for all deployment-specific secrets (database passwords, API keys, model endpoints), making the application trivially portable across development, staging, and production environments.

5.2.3 Shared Schema: The System Lingua Franca

`core/schema/wf_state.py` defines the data contracts that all modules use to communicate. By agreeing on a single, versioned set of Pydantic models, all modules interoperate without knowing each other's internals. The key classes are described in Table 5.2.

Table 5.2: Shared Schema Classes in `core/schema/wf_state.py`

Class	Role and Key Fields
<code>ConceptNode</code>	Atomic EKG unit at the application layer. Carries <code>name</code> , <code>out_degree</code> (semantic centrality from Section ??), <code>course_name</code> , and <code>mastery</code> (0–6 on Bloom's Taxonomy).
<code>StudentState</code>	<code>TypedDict</code> for the complete cognitive state: <code>current_pos</code> , <code>upcoming_nodes</code> (roadmap), <code>mastery_map</code> (concept→score), and <code>pending_proposal</code> (unconfirmed path update).
<code>AgentState</code>	LangGraph state envelope flowing through the agentic workflow. Encapsulates <code>messages</code> , <code>classified intent</code> , <code>accumulated worker_results</code> , and a <code>ui_action</code> field for frontend navigation directives.
<code>TAOutput</code>	Structured, serializable response from the TA workflow to the API controller. Contains <code>summary</code> (memo heading), <code>message</code> (displayed to student), and optional <code>ui_action</code> .

5.2.4 Repository Clients: Abstracted Database Access

The `core/repo/` sub-package provides repository wrapper classes for each database engine: `GraphDB` (Neo4j), `MilvusDB` (vector search), `MinioDB` (object storage), `Mongo_DB` (document store), and `SQL_DB` (relational). Each wrapper encapsulates the engine-specific client library and exposes a clean, high-level interface to the domain modules.

Critically, no domain module ever imports a database client directly. All repository clients are received exclusively as constructor parameters injected by the lifespan initializer. This strict application of the Dependency Inversion Principle means that switching the underlying database technology requires modifying only the configuration dataclass and the lifespan constructor, without touching a single line of business logic in any domain module.

5.3 Student Module

5.3.1 Authentication: OAuth2 + JWT

The student module implements a standard OAuth2 password flow using `PyJWT` for JWT minting. The `/student/login` endpoint validates credentials against SHA-256 hashed passwords in SQLite and returns

a short-lived JWT (sub = student_id). All subsequent requests attach this token as a Bearer credential; `get_current_student()` (a FastAPI dependency) decodes and validates it on each call.

5.3.2 Session Lifecycle and the Identity Firewall

A critical design decision in the system is the *identity firewall*: the TA Module must never learn which student it is teaching. This is enforced architecturally:

1. After login, the student calls `POST /student/session/start` with their JWT. The server generates a random UUID (`session_id`) and registers it in `Student_Tracker`.
2. All subsequent TA interactions supply only `session_id`. The TA Module never receives `student_id`.
3. Inside `Student_Tracker._resolve()`, the private mapping `session_id → student_id` translates transparently before any database call.

This pattern is analogous to a hotel key-card system: the key (`session_id`) grants access to resources, but revealing it does not expose the guest's identity (`student_id`).

5.3.3 Student_Tracker: Multi-Session Isolation and Shared State

As discussed in Section 5.1, `Student_Tracker` acts as a Facade over four persistence systems. In the current implementation, it manages two distinct layers of state:

1. **Session-Specific State (`StudentSession`):** Keyed by `session_id`, this layer isolates chat history and session-specific context (like recent PDF pages). This prevents "Identity Race Conditions" where multiple browser tabs contaminate each other's chat logs.
2. **Shared Learning State:** Keyed by `student_id`, this layer caches the student's learning progress (current position, mastery map, etc.) and is shared across all active sessions of the same student.

To ensure data integrity during concurrent updates, the tracker utilizes **Atomic Field Updates** in MongoDB (using the `$set` and `$addToSet` operators). Instead of overwriting the entire state object, the system updates specific fields (e.g., updating only the `current_pos` or appending to `finished_communities`). This design effectively mitigates "Data Race Conditions" where one session might accidentally revert progress made in another concurrent session.

5.4 Knowledge Module

The Knowledge Module operates as an offline, administrative pipeline responsible for constructing the Educational Knowledge Graph that the TA Module consumes at runtime. It exposes a REST interface for course ingestion triggered by administrators, not students.

5.4.1 Ingestion Pipeline

The pipeline proceeds through three stages: (1) Extraction — raw PDF pages are read from MinIO in configurable batch sizes (`PAGE_PER_SLIDE=15`); (2) Transformation — the `CoreLLMEngine` is invoked with an information-extraction prompt to produce semantic triplets; (3) Loading — extracted entities and relationships are written to Neo4j and vector embeddings are indexed in Milvus using the `allenai/scibert_scivocab_uncased` model (768-dimensional, domain-appropriate for scientific text).

5.4.2 Graph Schema

Nodes in Neo4j carry a `rrole` property indicating their rhetorical role (Definition, Statement, Application), derived from the `RhetoricalRetriever` tool's taxonomy. This role is used at retrieval time to match the nature of the student's query ("what is" → Definition; "how does it work" → Statement). Community membership is stored as a node property, enabling `course_backbone` queries to retrieve the skeletal hub nodes of a course in a single graph traversal.

5.5 Teaching Assistant (TA) Module

The TA Module is the system's most architecturally complex component. It orchestrates a multi-agent reasoning workflow that processes student queries, adapts to their learning state, and produces educationally grounded responses.

5.5.1 Module-Level Design: Stateless Singleton with External Context

TAModule is instantiated once at startup and shared across all concurrent requests. However, it must serve many distinct students simultaneously. This is resolved by a strict statelessness contract: TAModule holds *no per-student data*. All transient state is injected at call time through two channels:

1. **Student Tracker (Facade injection):** passed to `execute()` and threaded through LangGraph's configurable dictionary, making it accessible to any node in the graph without polluting AgentState.
2. **Session ID:** the opaque token that the Student Tracker uses to resolve student identity internally.

This mirrors the Stateless Service pattern used in microservices architectures: the service itself is a pure function of its inputs.

5.5.2 Agent Injection: Factory + Strategy Patterns

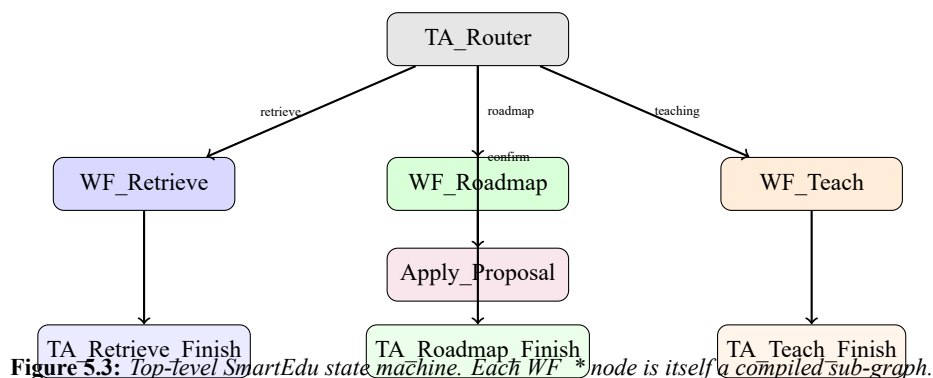
`AgentInjector.initialize_all_agents()` implements the *Factory* pattern. Given a configuration list of (agent_name, system_prompt, output_schema) tuples, it constructs each agent as a LangChain runnable chain:

```
1 chain = prompt_template | llm_with_tools
```

The LLM instance is retrieved from CoreLLMEngine (Multiton), and tools are retrieved from ToolFactory by agent name. The resulting chain is stored in an agents dict and passed to SmartEdu and the workflow builders. This is also a *Strategy* pattern: swapping an agent's reasoning strategy requires only changing the system prompt or tool list in the configuration, with no structural code change.

5.5.3 SmartEdu: State Machine (LangGraph)

The top-level SmartEdu class builds a LangGraph StateGraph that acts as a hierarchical state machine. The *State Machine* pattern is highly appropriate here because the teaching workflow has a finite, well-defined set of modes (Retrieve, Roadmap, Teach, Confirm, Clarify) with deterministic transitions between them.



The router node uses `wrap_agent_structured(agent, 0.0, RouterDecision)` to obtain a Pydantic-validated RouterDecision from the LLM, eliminating string parsing and ensuring deterministic branching.

5.5.4 Teaching Sub-Graph: Determinism and RAG Fallback

The `build_teach_wf()` sub-graph introduces a key design innovation: separating *data retrieval* from *content generation*, with a deterministic Python-level fallback.

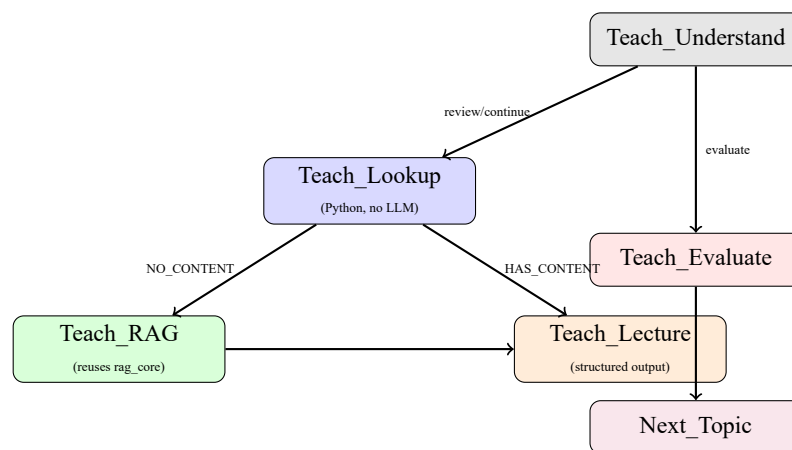


Figure 5.4: Teaching sub-graph. The conditional edge from *Teach_Lookup* is resolved by a Python function, not an LLM call.

Teach_Lookup calls `GetConcept._run()` and `GetPages._run()` directly as Python functions, bypassing the LLM entirely. This deterministic lookup produces either a `source=PDF` context block or a `source=NO_CONTENT` signal. The conditional edge `_route_after_lookup()` branches based on this Python value — a critical reliability improvement over instruction-following LLM prompts for structural decisions.

Teach_RAG reuses `rag_core()` from `workflow/retrieve.py`, constructing a synthetic query about the target concept. This eliminates code duplication between the Retrieve and Teach workflows and ensures that the fallback has identical retrieval quality to the primary RAG path.

Teach_Lecture receives pre-fetched content through `_teach_context` in `AgentState` and generates a structured `TeachLectureOutput` (Pydantic). Because the data is already retrieved, the LLM’s role is purely generative, eliminating the need for tool calls during lecture generation and reducing hallucination risk.

5.5.5 Prompt Management: Module-Level `__getattr__`

`TA/edu/workflow/prompt.py` implements a sophisticated lazy-loading pattern using Python’s module-level `__getattr__` hook. All prompt strings are defined with a leading underscore (e.g., `_ROUTER_PROMPT`), making them internal. When external code imports `ROUTER_PROMPT`, the `__getattr__` intercept calls `PromptManager.get_prompt()`, which first attempts to fetch the prompt from Langfuse Prompt Management (allowing runtime updates without redeployment) and falls back to the local underscore-prefixed variable if unavailable. This implements the *Proxy* pattern: the caller sees a simple module attribute, while the proxy transparently adds remote-fetch and fallback logic.

5.6 Tracing and Observability Module

`AgentTracer` is a per-session observer that buffers reasoning steps in memory and flushes them asynchronously to a JSON file at the end of each chat turn. It implements the *Observer* pattern: the TA Module calls `log_step()` at each graph node without blocking execution. The `end_chat()` coroutine then serialises the complete `ChatTrace` and optionally flushes telemetry to Langfuse.

Langfuse integration is opt-in and fail-safe: if environment variables `LANGFUSE_SECRET_KEY` and `LANGFUSE_PUBLIC_KEY` are absent, the tracer initialises silently without Langfuse. If the `langfuse` package is not installed, a warning is logged and execution continues. This *Null Object* approach ensures that observability infrastructure never degrades core functionality.

5.7 Database Design and Management

The system employs a *polyglot persistence* strategy, selecting each database engine for the data access patterns of its specific use case:

Table 5.3: *Polyglot Persistence Strategy*

Database	Data	Primary Access Pattern
Neo4j (Knowledge)	Concepts, relationships, communities	Graph traversal (Cypher)
Neo4j (Student)	Mastery edges, learning history	Ego-graph lookup by student
Milvus	Concept embeddings (768-dim)	ANN similarity search
MinIO	Raw PDFs, lecture slides	Object get by URI + page
MongoDB	Student state, chat history	Document upsert by student_id
SQLite	Student identity, credentials	Primary-key lookup + auth

All database clients are injected as constructor parameters into the modules that need them. No module imports a database client directly; they receive it from the lifespan initialiser. This dependency inversion means that the database implementation can be swapped (e.g., from local Neo4j to AuraDB) by changing only the configuration and the lifespan factory, with zero changes to business logic.

Bibliography

- [1] Qurat Ul Ain, Mohamed Amine Chatti, Jean Qussa, Amr Shakhshir, Rawaa Alatrash, et al. The edukg-pipeline: An optimized pipeline for automatic educational knowledge graph construction. *arXiv preprint arXiv:2509.05392*, 2025.
- [2] Sydney Anuyah, Mehedi Mahmud Kaushik, Krishna Dwarampudi, Rakesh Shiradkar, Arjan Durrezi, and Sunandan Chakraborty. Automated knowledge graph construction using large language models and sentence complexity modelling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- [3] Joao T. Aparicio, Elisabete Arsenio, Francisco Santos, and Rui Henriques. Using dynamic knowledge graphs to detect emerging communities of knowledge. *Knowledge-Based Systems*, 2024.
- [4] Docker. Docker overview, 2023.
- [5] Haoyu Huang, Chong Chen, Zeang Sheng, Yang Li, and Wentao Zhang. Can llms be good graph judge for knowledge graph construction?, 2025.
- [6] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. Stanford University, 3rd edition, 2026. Online manuscript released January 6, 2026.
- [7] Chen Liang, Jianbo Ye, Zhaohui Wu, Bart Pursel, and C Lee Giles. Recovering concept prerequisite relations from university course dependencies. In *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.
- [8] Chenxi Liu, Yongqiang Chen, Tongliang Liu, James Cheng, Bo Han, and Kun Zhang. On the thinking-language modeling gap in large language models. *arXiv preprint arXiv:2505.12896*, 2025.
- [9] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.
- [10] Congmin Min, Sahil Bansal, Joyce Pan, Abbas Keshavarzi, Rhea Mathew, and Amar Viswanathan Kanan. Towards practical graphrag: Efficient knowledge graph construction and hybrid retrieval at scale. *arXiv:2507.03226*, 2025.
- [11] Belinda Mo, Kyssen Yu, Joshua Kazdan, Joan Cabezas, Proud Mpala, Lisa Yu, Chris Cundy, Charilaos Kanatsoulis, and Sanmi Koyejo. Kggen: Extracting knowledge graphs from plain text with language models, 2025.
- [12] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [13] Liangming Pan, Chengjiang Li, Juanzi Li, and Jie Tang. Prerequisite relation learning for concepts in moocs. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017.
- [14] Simplilearn. What is aws? an ultimate guide to amazon web services, 2023.
- [15] Sharon Solomon. Frontegg - what is oauth? definition, how it works, and tips for success, 2023.
- [16] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9, 2019.
- [17] Jing Wang, Zheng Li, Lei Li, Fan He, Liyu Lin, Yao Lai, Yan Li, Xiaoyang Zeng, and Yufeng Guo. Principle-guided verillog optimization: Ip-safe knowledge transfer via local-cloud collaboration. *10.48550/arXiv.2508.05675*, 2025.

- [18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [19] Orion Weller, Michael Boratko, Iftekhhar Naim, and Jinhyuk Lee. On the theoretical limitations of embedding-based retrieval. *arXiv preprint arXiv:2508.21038*, 2025.
- [20] Bowen Zhang and Harold Soh. Extract, define, canonicalize: An LLM-based framework for knowledge graph construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9820–9836, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- [21] B. Zhao, J. Sun, B. Xu, X. Lu, Y. Li, J. Yu, M. Liu, T. Zhang, Q. Chen, H. Li, et al. Edukg: a heterogeneous sustainable k-12 educational knowledge graph. *arXiv preprint arXiv:2210.12228*, 2022.